JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 03 OF 17 - STUDY STEP 03B

Number Systems Interview Patterns and Rapid Revision

Part B: Fifty-Two Implementations, Java Traps, Exercises, Solutions, and Readiness

BY

Vinay Reddy Kalluri

Convert the numeric foundation into reusable interview patterns, boundary-safe Java, debugging skill, timed practice, and rapid revision.

52 IMPLEMENTATIONS | JAVA TRAPS | DEBUGGING | SOLUTIONS | REVISION

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 02 — Number Systems and Math Foundations`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This linked workbook completes Study Step 03 with the 52-implementation catalog, Java number traps, conceptual and code-output questions, debugging tasks, coding exercises, solutions, follow-up chains, cheat sheet, and readiness assessment.

Readiness map

Decision	Evidence
START HERE	A number concept is understood but implementation is not yet automatic; Boundary bugs appear during timed coding; The interviewer changes the range, sign, base, or overflow contract; Final revision needs retrieval and debugging rather than more passive reading.
PREPARE FIRST	Study Step 03A numeric foundations; Ability to write and run small Java 21 methods; An error log for boundary, overflow, and contract mistakes.
MOVE ON WHEN	Implement and explain all 52 mandatory number algorithms; Diagnose Java numeric traps from failing inputs; Complete short and medium problems under explicit contracts; Use the readiness assessment to decide when to advance to bit manipulation.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 02 - Number Systems and Math Foundations	**YOU ARE HERE** **DSA 03 - Number Systems Interview Workbook**	DSA 04 - Bit Manipulation

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Thirty Core Interview Problem Patterns

Interview problems involving numbers become much easier when they are grouped by invariant instead of memorized as unrelated tricks. Digit problems repeatedly remove a base-10 digit. Base-conversion problems repeatedly multiply and add, or repeatedly divide and collect remainders. Factor problems search only through a square-root boundary. Large-number-string problems carry a small state while scanning characters. Overflow-safe problems validate an operation before trusting its fixed-width result.

This chapter is a catalog of thirty reusable core patterns and four closely related extensions. Chapter 14A expands the compiling companion to all fifty-two required implementations while preserving this shorter high-frequency learning route.

14.1 How to use the catalog

For each numbered pattern, rehearse this sequence:

1. State the input contract.
2. Name the recognition signal.
3. Describe the natural brute-force idea.
4. Improve it by identifying the invariant.
5. Write the optimal interview implementation.
6. Dry-run one normal case and one boundary.
7. State time and space complexity.
8. Name the failure modes before the interviewer asks.
9. Offer a realistic follow-up.

The implementations use Java 21 syntax but avoid distracting language features. They throw IllegalArgumentException for malformed input unless the common coding-platform contract calls for a sentinel, such as returning zero when a reversed int overflows.

14.2 Coverage map

ID	Mandatory problem	Primary method
1	Count digits	DigitPatterns.countDigits
2	Sum of digits	DigitPatterns.sumDigits
3	Reverse an integer safely	DigitPatterns.reverseIntOrZero
4	Palindrome number	DigitPatterns.isPalindrome
5	Armstrong number	DigitPatterns.isArmstrong
6	Strong number	DigitPatterns.isStrong
7	Binary string to decimal	BaseConversionPatterns.binaryToLong
8	Decimal to binary	BaseConversionPatterns.toBinary
9	Generic base to decimal	BaseConversionPatterns.baseToLong
10	Decimal to generic base	BaseConversionPatterns.longToBase
11	Large number divisible by 9	LargeNumberPatterns.isDivisibleBy9
12	Large number divisible by 11	LargeNumberPatterns.isDivisibleBy11
13	Prime check	NumericPropertyPatterns.isPrime
14	Print factors	NumericPropertyPatterns.factors
15	Count factors	NumericPropertyPatterns.factorCount
16	Prime factorization	NumericPropertyPatterns.primeFactors
17	GCD	NumericPropertyPatterns.gcd
18	LCM	NumericPropertyPatterns.safeLcm
19	Perfect square	NumericPropertyPatterns.isPerfectSquare
20	Integer square root	NumericPropertyPatterns.floorSquareRoot
21	Power of two	NumericPropertyPatterns.isPowerOfTwo
22	Fast exponentiation	NumericPropertyPatterns.safePower
23	Large numeric string modulo	LargeNumberPatterns.modulo
24	Add two numeric strings	LargeNumberPatterns.add
25	Compare two numeric strings	LargeNumberPatterns.compare
26	Safe multiplication	OverflowPatterns.safeMultiply

ID	Mandatory problem	Primary method
27	Overflow-safe midpoint	OverflowPatterns.safeMidpoint
28	Normalize negative modulo	NumericPropertyPatterns.normalizeModulo
29	Hexadecimal string to decimal	BaseConversionPatterns.hexToLong
30	Decimal to hexadecimal	BaseConversionPatterns.toHex

14.3 Shared contracts

The catalog makes these contracts explicit:

- A decimal digit problem treats zero as a one-digit value.
- Digit aggregation uses the magnitude of a signed int, promoted to long before absolute value.
- Base-conversion strings represent nonnegative values and use bases 2 through 36.
- Generic digits use 0-9 and A-Z; input accepts either letter case.
- Large-number strings contain decimal digits only. Leading zeros are valid.
- Factor and LCM methods accept nonnegative values as documented.
- A modulus must be positive.
- A failed exact fixed-width operation is represented with OptionalLong.
- Java code never assumes that assigning an overflowed int expression to long repairs it.

WHY IT WORKS | 14.4 Digit-traversal invariant

For a nonnegative decimal value n:

TEXT

```
last digit       = n % 10
remaining prefix = n / 10
```

Every iteration removes one digit, so a d-digit number takes O(d) time, which is O(log10(n)) for positive n. Interviewers often accept O(number of digits), which is the clearer statement.

Pattern 1: Count digits

Recognition signal: The task asks for decimal length without converting to a string, or a later rule needs the number of digits.

Brute force: Convert to a string and count characters, adjusting for a sign. This is easy but allocates a string.

Better and optimal approach: Promote to long, take the magnitude, divide by 10 until zero, and define zero as one digit.

Java solution: DigitPatterns.countDigits.

Complexity: O(d) time and O(1) space.

Dry run: 40,502 becomes 4,050, 405, 40, 4, 0, so the count is 5.

Edge cases: Zero, negative input, and Integer.MIN_VALUE.

Common mistake: Math.abs(Integer.MIN_VALUE) remains negative. Promote before taking the magnitude.

Follow-up: Count digits in an arbitrary base b by repeatedly dividing by b.

Pattern 2: Sum of digits

Recognition signal: Divisibility rules, repeated digit transformation, digital-root questions, or a checksum based on decimal digits.

Brute force: Convert to text and parse each one-character substring.

Better and optimal approach: Repeatedly add magnitude % 10 and divide the magnitude by 10.

Java solution: DigitPatterns.sumDigits.

Complexity: O(d) time and O(1) space.

Dry run: 5,074 produces 4 + 7 + 0 + 5 = 16.

Edge cases: The sum of digits of zero is zero; the sign is not a digit.

Common mistake: Letting a negative remainder contaminate the sum.

Follow-up: Repeatedly sum until one digit remains, then derive the digital-root formula.

Pattern 3: Reverse an integer safely

Recognition signal: Digits must be reconstructed in reverse order under a fixed-width return type.

Brute force: Convert to a string, reverse it, parse it, and catch parsing overflow.

Better approach: Rebuild numerically with reversed = reversed * 10 + digit using a wider long.

Optimal interview approach: For an int input, a long accumulator is sufficient because the reversed magnitude has at most ten decimal digits. Apply the sign and range-check before narrowing. A stricter variant can guard before every multiply-add.

Java solution: DigitPatterns.reverseIntOrZero.

Complexity: O(d) time and O(1) space.

Dry run: -120 becomes digits 0, 2, 1; the magnitude rebuilds as 21 and the result is -21.

Edge cases: Zero, trailing zeros, Integer.MIN_VALUE, and a reversed value outside the int range.

Common mistake: Reversing in int and checking after overflow has already happened.

Follow-up: Return OptionalInt instead of the coding-platform sentinel zero.

Pattern 4: Palindrome number

Recognition signal: Decimal digits must read identically from both ends.

Brute force: Convert to a string and compare mirrored characters.

Better approach: Reverse the entire number numerically and compare, but that can overflow.

Optimal interview approach: Reverse only the lower half of the digits. Stop when the original prefix is no larger than the reversed suffix.

Java solution: DigitPatterns.isPalindrome.

Complexity: O(d) time and O(1) space.

Dry run: 12,321 evolves from prefix 12,321 and suffix 0 to prefix 12 and suffix 123. For an odd digit count, remove suffix's middle digit: 12 == 123 / 10.

Edge cases: Negative values are false. Zero is true. A positive value ending in zero is false unless the value is zero.

Common mistake: Reversing the full int and introducing overflow.

Follow-up: Test a palindrome in base b.

Pattern 5: Armstrong number

An Armstrong number equals the sum of each decimal digit raised to the number of digits. For example, 153 = 1^3 + 5^3 + 3^3.

Recognition signal: The property combines digit count with per-digit exponentiation.

Brute force: Convert to a string, use Math.pow on each character, and cast from double.

Better and optimal approach: Count digits once, traverse digits, and compute each small integer power with integer multiplication.

Java solution: DigitPatterns.isArmstrong.

Complexity: O(d^2) under the simple repeated-multiplication helper, or O(d log d) with exponentiation by squaring. Since int has at most ten digits, either is bounded and clear.

Dry run: 153 has three digits; 1 + 125 + 27 = 153.

Edge cases: This implementation defines the property only for nonnegative ints. Zero is an Armstrong number.

Common mistake: Trusting floating-point Math.pow for an exact integer equality.

Follow-up: Generalize to base b and state how the digit count changes.

Pattern 6: Strong number

A Strong number equals the sum of the factorials of its decimal digits. For example, 145 = 1! + 4! + 5!.

Recognition signal: Each digit maps to a small reusable value from 0! through 9!.

Brute force: Recompute each factorial for every occurrence.

Better and optimal approach: Precompute the ten factorials once and perform a digit traversal.

Java solution: DigitPatterns.isStrong.

Complexity: O(d) time and O(1) extra space because the table has fixed size ten.

Dry run: 145 produces 1 + 24 + 120 = 145.

Edge cases: 0! is 1, so zero itself is not Strong under this definition. Negative values are rejected as false.

Common mistake: Initializing 0! to zero.

Follow-up: Find every Strong number in a range without recomputing factorials.

14.5 Compiling digit-pattern implementation

JAVA (continued 1/4)

```java
public final class DigitPatterns {
    private static final int[] DIGIT_FACTORIAL = {
        1, 1, 2, 6, 24, 120, 720, 5_040, 40_320, 362_880
    };

    private DigitPatterns() {
    }

    public static int countDigits(int value) {
        long remaining = Math.abs((long) value);
        int count = 1;
        while (remaining >= 10) {
            remaining /= 10;
            count++;
        }
        return count;
    }

    public static int sumDigits(int value) {
        long remaining = Math.abs((long) value);
        int sum = 0;
        do {
            sum += (int) (remaining % 10);
            remaining /= 10;
        } while (remaining > 0);
        return sum;
    }

    public static long productDigits(int value) {
```

JAVA (continued 2/4)

```
        long remaining = Math.abs((long) value);
        long product = 1;
        do {
            product *= remaining % 10;
            remaining /= 10;
        } while (remaining > 0);
        return product;
    }

    public static int reverseIntOrZero(int value) {
        long remaining = Math.abs((long) value);
        long reversed = 0;
        do {
            reversed = reversed * 10 + remaining % 10;
            remaining /= 10;
        } while (remaining > 0);

        long signed = value < 0 ? -reversed : reversed;
        if (signed < Integer.MIN_VALUE || signed > Integer.MAX_VALUE) {
            return 0;
        }
        return (int) signed;
    }

    public static boolean isPalindrome(int value) {
        if (value < 0 || value != 0 && value % 10 == 0) {
            return false;
        }
```

JAVA (continued 3/4)

```
        int prefix = value;
        int reversedSuffix = 0;
        while (prefix > reversedSuffix) {
            reversedSuffix = reversedSuffix * 10 + prefix % 10;
            prefix /= 10;
        }
        return prefix == reversedSuffix
                || prefix == reversedSuffix / 10;
    }

    public static boolean isArmstrong(int value) {
        if (value < 0) {
            return false;
        }
        int digits = countDigits(value);
        int remaining = value;
        long sum = 0;
        do {
            int digit = remaining % 10;
            sum += integerPower(digit, digits);
            remaining /= 10;
        } while (remaining > 0);
        return sum == value;
    }

    public static boolean isStrong(int value) {
        if (value < 0) {
            return false;
```

JAVA (continued 4/4)

```
        }
        int remaining = value;
        int sum = 0;
        do {
            sum += DIGIT_FACTORIAL[remaining % 10];
            remaining /= 10;
        } while (remaining > 0);
        return sum == value;
    }

    private static long integerPower(int base, int exponent) {
        long result = 1;
        for (int i = 0; i < exponent; i++) {
            result *= base;
        }
        return result;
    }

    public static void main(String[] args) {
        System.out.println(countDigits(Integer.MIN_VALUE)); // 10
        System.out.println(sumDigits(-5_074));              // 16
        System.out.println(productDigits(1_234));           // 24
        System.out.println(reverseIntOrZero(-120));         // -21
        System.out.println(isPalindrome(12_321));           // true
        System.out.println(isArmstrong(153));               // true
        System.out.println(isStrong(145));                  // true
    }
}
```

Extension: Product of digits

Product of digits uses the same traversal. Zero needs deliberate handling: the number zero has one digit, zero, so its digit product is zero. Any number containing an internal zero also has product zero.

Recognition signal: A property multiplies independent decimal digits.

Complexity: O(d) time and O(1) space.

Follow-up: Decide how to detect overflow if the input is a very long decimal string rather than an int.

WHY IT WORKS | 14.6 Base-conversion invariants

Converting from a base-b string to decimal uses Horner accumulation:

TEXT

```
result = result * base + nextDigit
```

Converting a nonnegative decimal value to base b uses repeated division:

TEXT

```
digit     = value % base
remaining = value / base
```

Remainders arrive from least significant to most significant, so they are reversed at the end.

Pattern 7: Binary string to decimal

Recognition signal: A sequence of characters '0' and '1' represents a numeric value, not a decimal text.

Brute force: For each one bit, call Math.pow(2, position) and add a cast result.

Better approach: Scan right to left with a power variable, checking overflow.

Optimal interview approach: Scan left to right and apply result = result * 2 + bit, checking before each update.

Java solution: BaseConversionPatterns.binaryToLong.

Complexity: O(d) time and O(1) space.

Dry run: "1011" accumulates 1, 2, 5, 11.

Edge cases: Empty input, nonbinary characters, leading zeros, and a value larger than Long.MAX_VALUE.

Common mistake: Parsing through int first when a long result was promised.

Follow-up: Return the value modulo m without requiring the full value to fit.

Pattern 8: Decimal to binary

Recognition signal: A nonnegative integer must be represented in base 2 without a library formatter.

Brute force: Find the largest power of two and test every position.

Better and optimal approach: Repeatedly append value % 2, divide by 2, and reverse.

Java solution: BaseConversionPatterns.toBinary.

Complexity: O(log2(value)) time and O(log2(value)) output space.

Dry run: 13 produces remainders 1, 0, 1, 1; reversing gives "1101".

Edge cases: Zero must return "0". This contract accepts nonnegative long values only.

Common mistake: Returning an empty string for zero.

Follow-up: Emit exactly 64 bits, including leading zeros.

Pattern 9: Generic base to decimal

Recognition signal: The input provides digits and a radix, often from 2 through 36.

Brute force: Multiply each digit by Math.pow(base, position).

Better approach: Maintain an integer positional power from right to left.

Optimal interview approach: Horner accumulation needs one multiply-add per digit and no separate power.

Java solution: BaseConversionPatterns.baseToLong.

Complexity: O(d) time and O(1) space.

Dry run: "2A" in base 16 accumulates 2, then 2 * 16 + 10 = 42.

Edge cases: Invalid base, invalid digit, mixed letter case, leading zeros, empty input, and overflow.

Common mistake: Accepting Character.digit(ch, base) == -1 as a real digit.

Follow-up: Support an optional sign without allowing a sign-only string.

Pattern 10: Decimal to generic base

Recognition signal: A nonnegative fixed-width value must be formatted in a requested radix.

Brute force: Repeatedly search for the largest power of the base.

Better and optimal approach: Collect repeated remainders and reverse them.

Java solution: BaseConversionPatterns.longToBase.

Complexity: O(log_base(value)) time and the same amount of output space.

Dry run: 42 in base 16 produces remainders 10 and 2, which map to A and 2; reverse to "2A".

Edge cases: Zero, invalid bases, and the chosen nonnegative-input contract.

Common mistake: Mapping remainder 10 to the two characters "10" instead of digit A.

Follow-up: Support negative values, including Long.MIN_VALUE, without calling Math.abs on it.

Pattern 29: Hexadecimal string to decimal

Recognition signal: Digits may include A-F and represent powers of sixteen.

Brute force: Use a position-by-position power formula with Math.pow.

Better and optimal approach: Reuse generic Horner accumulation with base 16.

Java solution: BaseConversionPatterns.hexToLong.

Complexity: O(d) time and O(1) extra space.

Dry run: "7F" accumulates 7, then 7 * 16 + 15 = 127.

Edge cases: Letter case, optional prefix policy, invalid G-Z digits, and overflow. This method deliberately does not accept a 0x prefix.

Common mistake: Subtracting '0' from a letter digit.

Follow-up: Accept and validate an optional 0x or 0X prefix.

Pattern 30: Decimal to hexadecimal

Recognition signal: A nonnegative value must be rendered compactly in base 16.

Brute force: Repeatedly compare against a hard-coded table of powers of sixteen.

Better and optimal approach: Reuse generic repeated division with base 16.

Java solution: BaseConversionPatterns.toHex.

Complexity: O(log16(value)) time and output space.

Dry run: 255 produces remainders 15 and 15, giving "FF".

Edge cases: Zero and the selected uppercase/lowercase convention.

Common mistake: Forgetting that the returned representation is text, not a decimal integer containing hexadecimal digits.

Follow-up: Produce Java-style fixed-width two's-complement output for a negative int.

Extension: Validate a binary string

Validation is part of parsing, not an optional cleanup step. A valid nonnegative binary string is non-null, nonempty, and contains only '0' or '1'. Decide separately whether whitespace, signs, separators, or a 0b prefix are allowed. Silently deleting invalid characters changes the input and is rarely an acceptable interview contract.

14.7 Compiling base-conversion implementation

JAVA (continued 1/4)

```java
public final class BaseConversionPatterns {
    private static final char[] DIGITS =
            "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();

    private BaseConversionPatterns() {
    }

    public static boolean isValidBinary(String text) {
        if (text == null || text.isEmpty()) {
            return false;
        }
        for (int i = 0; i < text.length(); i++) {
            char current = text.charAt(i);
            if (current != '0' && current != '1') {
                return false;
            }
        }
        return true;
    }

    public static long binaryToLong(String binary) {
        if (!isValidBinary(binary)) {
            throw new IllegalArgumentException("invalid binary string");
        }
        return baseToLong(binary, 2);
    }

    public static String toBinary(long value) {
```

JAVA (continued 2/4)

```
        return longToBase(value, 2);
    }

    public static long hexToLong(String hexadecimal) {
        return baseToLong(hexadecimal, 16);
    }

    public static String toHex(long value) {
        return longToBase(value, 16);
    }

    public static long baseToLong(String text, int base) {
        requireBase(base);
        if (text == null || text.isEmpty()) {
            throw new IllegalArgumentException("digits must not be empty");
        }

        long result = 0;
        for (int i = 0; i < text.length(); i++) {
            int digit = asciiDigit(text.charAt(i));
            if (digit < 0 || digit >= base) {
                throw new IllegalArgumentException(
                        "invalid digit at index " + i);
            }
            if (result > (Long.MAX_VALUE - digit) / base) {
                throw new ArithmeticException("value exceeds long range");
            }
            result = result * base + digit;
```

JAVA (continued 3/4)

```
        }
        return result;
    }

    public static String longToBase(long value, int base) {
        requireBase(base);
        if (value < 0) {
            throw new IllegalArgumentException(
                    "value must be nonnegative");
        }
        if (value == 0) {
            return "0";
        }

        StringBuilder reversed = new StringBuilder();
        long remaining = value;
        while (remaining > 0) {
            int digit = (int) (remaining % base);
            reversed.append(DIGITS[digit]);
            remaining /= base;
        }
        return reversed.reverse().toString();
    }

    private static void requireBase(int base) {
        if (base < Character.MIN_RADIX
                || base > Character.MAX_RADIX) {
```

JAVA (continued 4/4)

```
            throw new IllegalArgumentException(
                    "base must be between 2 and 36");
        }
    }

    private static int asciiDigit(char value) {
        if (value >= '0' && value <= '9') {
            return value - '0';
        }
        if (value >= 'A' && value <= 'Z') {
            return value - 'A' + 10;
        }
        if (value >= 'a' && value <= 'z') {
            return value - 'a' + 10;
        }
        return -1;
    }

    public static void main(String[] args) {
        System.out.println(binaryToLong("001011")); // 11
        System.out.println(toBinary(13));           // 1101
        System.out.println(baseToLong("2A", 16));   // 42
        System.out.println(longToBase(42, 16));     // 2A
        System.out.println(hexToLong("7f"));        // 127
        System.out.println(toHex(255));             // FF
    }
}
```

The parser checks result > (Long.MAX_VALUE - digit) / base before multiplying. This guard proves that the next multiply-add fits. Checking result after overflow would be too late.

14.8 Numeric-property patterns

Many numeric properties become tractable when a search stops at sqrt(n). If n has a factor greater than its square root, the paired factor is smaller than the square root and has already been discovered.

Pattern 13: Prime check

Recognition signal: Determine whether a value has exactly two positive factors.

Brute force: Test every divisor from 2 through n - 1.

Better approach: Test only through sqrt(n).

Optimal interview approach: Reject values below two, handle two and even values, then test odd divisors while divisor <= n / divisor. Division avoids divisor * divisor overflow.

Java solution: NumericPropertyPatterns.isPrime.

Complexity: O(sqrt(n)) time and O(1) space.

Dry run: For 29, test 3 and 5; neither divides, and the next odd divisor 7 exceeds 29 / 7.

Edge cases: Negative values, 0, 1, 2, and large values near Long.MAX_VALUE.

Common mistake: Treating 1 as prime or writing divisor * divisor <= n in long.

Follow-up: Generate all primes through n with the Sieve of Eratosthenes.

Pattern 14: Print factors

Recognition signal: Enumerate all positive divisors, usually in sorted order.

Brute force: Test every candidate from 1 through n.

Better approach: Discover factor pairs through sqrt(n).

Optimal interview approach: Store small factors in forward order and paired large factors in reverse order, avoiding a duplicate when divisor == n / divisor.

Java solution: NumericPropertyPatterns.factors.

Complexity: O(sqrt(n)) search time, O(f) output space for f factors, and O(f) reversal time.

Dry run: For 36, pairs are (1,36), (2,18), (3,12), (4,9), and (6,6). Emit 6 only once.

Edge cases: This implementation requires n > 0. The factors of 1 are [1].

Common mistake: Printing both square-root partners and duplicating the root.

Follow-up: Return factors in sorted order without sorting the entire result.

Pattern 15: Count factors

Recognition signal: Only the number of divisors matters, not the list.

Brute force: Test 1 through n.

Better and optimal approach: Count two for each factor pair and one for a square-root pair.

Java solution: NumericPropertyPatterns.factorCount.

Complexity: O(sqrt(n)) time and O(1) space.

Dry run: 36 has four distinct pairs plus the pair (6,6), so the total is 2 + 2 + 2 + 2 + 1 = 9.

Edge cases: n must be positive; factorCount(1) is 1.

Common mistake: Adding two for a perfect-square root.

Follow-up: Use prime exponents: if n = p^a q^b, the count is (a + 1)(b + 1).

Pattern 16: Prime factorization

Recognition signal: Decompose a value into prime powers, often to derive divisor counts, GCDs, or multiplicative properties.

Brute force: Generate primes separately and repeatedly search the whole range.

Better approach: Trial-divide from two upward and remove each discovered factor completely.

Optimal interview approach for a single long: Remove factor two, test odd factors only while factor <= remaining / factor, then record any remaining value greater than one as prime.

Java solution: NumericPropertyPatterns.primeFactors.

Complexity: O(sqrt(n)) worst-case time and O(k) output space for k distinct primes.

Dry run: 84 removes 2 twice, then 3 once, leaving 7. The factorization is 2^2 * 3 * 7.

Edge cases: This method requires n > 0 and returns an empty map for 1.

Common mistake: Incrementing the divisor immediately after one division and missing repeated powers.

Follow-up: Factor many values by precomputing the smallest prime factor for every integer through a limit.

Pattern 17: Greatest common divisor

Recognition signal: Simplifying ratios, synchronizing cycles, reducing fractions, or finding a largest shared unit.

Brute force: Scan downward from min(a, b) until a common divisor appears.

Better and optimal approach: Euclid's algorithm repeatedly replaces (a, b) with (b, a % b) until b is zero.

Java solution: NumericPropertyPatterns.gcd.

Complexity: O(log min(a, b)) time and O(1) space.

Dry run: gcd(48, 18) moves through (18,12), (12,6), and (6,0), so the answer is 6.

Edge cases: gcd(0, b) is b; gcd(0, 0) is defined here as 0. Inputs must be nonnegative.

Common mistake: Applying Math.abs to Long.MIN_VALUE and assuming it became positive.

Follow-up: Return coefficients x and y satisfying ax + by = gcd(a, b).

Pattern 18: Least common multiple

Recognition signal: Find the first time cycles align or the smallest positive multiple shared by two values.

Brute force: Step through multiples of the larger input until one is shared.

Better approach: Use a * b / gcd(a, b), but multiplication may overflow before division.

Optimal interview approach: Divide first: (a / gcd(a, b)) * b, and verify the final multiplication.

Java solution: NumericPropertyPatterns.safeLcm.

Complexity: O(log min(a, b)) time for GCD and O(1) space.

Dry run: lcm(21, 6) uses gcd 3, reduced value 7, then 7 * 6 = 42.

Edge cases: If either input is zero, the result is zero. Inputs are nonnegative. An unrepresentable long result returns OptionalLong.empty.

Common mistake: Multiplying before dividing or ignoring the zero case.

Follow-up: Compute the LCM of an array while stopping on overflow.

Pattern 19: Perfect square

Recognition signal: Determine whether n equals k^2 for an integer k.

Brute force: Test every k from zero through n.

Better approach: Estimate with Math.sqrt and verify.

Optimal proof-friendly approach: Binary-search the integer root and compare with division rather than mid * mid.

Java solution: NumericPropertyPatterns.isPerfectSquare.

Complexity: O(log n) time and O(1) space.

Dry run: floorSquareRoot(80) returns 8; 80 / 8 is 10, so 80 is not 8 squared.

Edge cases: Negative values are false; zero and one are true.

Common mistake: Comparing Math.sqrt(n) % 1 == 0 and assuming floating-point output is exact for every long.

Follow-up: Determine whether n is a perfect cube without multiplication overflow.

Pattern 20: Integer square root

Recognition signal: Return floor(sqrt(n)) without a floating-point answer.

Brute force: Increment a candidate until its square is too large.

Better and optimal approach: Binary-search the monotonic predicate candidate <= n / candidate.

Java solution: NumericPropertyPatterns.floorSquareRoot.

Complexity: O(log n) time and O(1) space.

Dry run: For 27, candidates 7, 3, 5, and 6 leave answer 5.

Edge cases: Negative input is rejected. Zero and one return themselves.

Common mistake: Allowing a midpoint or square multiplication to overflow.

Follow-up: Search for the kth root or return an answer within a decimal tolerance.

Pattern 21: Power of two

Recognition signal: A positive value should contain exactly one set bit.

Brute force: Generate powers of two until reaching or passing n.

Better approach: Repeatedly divide by two and reject an odd intermediate.

Optimal interview approach: Require n > 0 and test (n & (n - 1)) == 0.

Java solution: NumericPropertyPatterns.isPowerOfTwo.

Complexity: O(1) time and O(1) space for a fixed-width long.

Dry run: 16 is 10000 in binary and 15 is 01111; AND is zero.

Edge cases: Zero and negative values must be false.

Common mistake: Omitting the positive guard.

Follow-up: Return the next representable power of two without overflow.

Pattern 22: Fast exponentiation

Recognition signal: Compute base ^ exponent for a large nonnegative exponent.

Brute force: Multiply the result by base exponent times.

Better and optimal approach: Exponentiation by squaring uses the exponent's binary representation. Square the base each round and multiply it into the result only when the current exponent bit is one.

Java solution: NumericPropertyPatterns.safePower.

Complexity: O(log exponent) multiplications and O(1) space.

Dry run: 3 ^ 5 uses exponent bits 101: result becomes 3, base squares to 9, then base squares to 81 and result becomes 243.

Edge cases: Exponent zero returns one, including 0 ^ 0 under this programming contract. Negative exponents are rejected. Overflow returns OptionalLong.empty.

Common mistake: Squaring one extra time after the exponent becomes zero and reporting irrelevant overflow.

Follow-up: Compute base ^ exponent modulo m while avoiding multiplication overflow.

Pattern 28: Normalize negative modulo

Recognition signal: An index, clock position, or modular state must be in [0, modulus) even when the input is negative.

Brute force: Repeatedly add modulus until the value becomes nonnegative.

Better approach: Use ((value % modulus) + modulus) % modulus, but the addition deserves range reasoning.

Optimal Java approach: Use Math.floorMod(value, modulus) with a positive modulus.

Java solution: NumericPropertyPatterns.normalizeModulo.

Complexity: O(1) time and O(1) space.

Dry run: -13 with modulus 5 normalizes to 2.

Edge cases: The modulus must be positive.

Common mistake: Assuming Java's % always returns a nonnegative value.

Follow-up: Normalize a circular array index after an arbitrary signed jump.

Extension: Perfect number

A positive integer is perfect when the sum of its positive proper divisors equals the value. Six is perfect because $1 + 2 + 3 = 6$.

Recognition signal: Sum factor pairs while excluding n itself.

Optimal approach: Start with sum 1 for $n > 1$, inspect divisors through sqrt(n), and add both members of each pair without duplicating a square root.

Complexity: O(sqrt(n)) time and O(1) space.

Common mistake: Including n as its own proper divisor.

14.9 Compiling numeric-property implementation

JAVA (continued 1/7)

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.OptionalLong;

public final class NumericPropertyPatterns {
    private NumericPropertyPatterns() {
    }

    public static boolean isPrime(long value) {
        if (value < 2) {
            return false;
        }
        if (value == 2) {
            return true;
        }
        if (value % 2 == 0) {
            return false;
        }
        for (long divisor = 3;
                divisor <= value / divisor;
                divisor += 2) {
            if (value % divisor == 0) {
                return false;
            }
        }
        return true;
    }

    public static List<Long> factors(long value) {
        requirePositive(value);
        List<Long> lower = new ArrayList<>();
        List<Long> upper = new ArrayList<>();
        for (long divisor = 1;
```

JAVA (continued 2/7)

```
                divisor <= value / divisor;
                divisor++) {
            if (value % divisor == 0) {
                lower.add(divisor);
                long paired = value / divisor;
                if (paired != divisor) {
                    upper.add(paired);
                }
            }
        }
        Collections.reverse(upper);
        lower.addAll(upper);
        return List.copyOf(lower);
    }

    public static int factorCount(long value) {
        requirePositive(value);
        int count = 0;
        for (long divisor = 1;
                divisor <= value / divisor;
                divisor++) {
            if (value % divisor == 0) {
                count += divisor == value / divisor ? 1 : 2;
            }
        }
        return count;
    }

    public static Map<Long, Integer> primeFactors(long value) {
        requirePositive(value);
        Map<Long, Integer> result = new LinkedHashMap<>();
        long remaining = value;

        while (remaining % 2 == 0 && remaining > 1) {
            result.merge(2L, 1, Integer::sum);
```

JAVA (continued 3/7)

```
            remaining /= 2;
        }
        for (long factor = 3;
                factor <= remaining / factor;
                factor += 2) {
            while (remaining % factor == 0) {
                result.merge(factor, 1, Integer::sum);
                remaining /= factor;
            }
        }
        if (remaining > 1) {
            result.merge(remaining, 1, Integer::sum);
        }
        return Map.copyOf(result);
    }

    public static long gcd(long first, long second) {
        requireNonnegative(first, "first");
        requireNonnegative(second, "second");
        long a = first;
        long b = second;
        while (b != 0) {
            long remainder = a % b;
            a = b;
            b = remainder;
        }
        return a;
    }

    public static OptionalLong safeLcm(long first, long second) {
        requireNonnegative(first, "first");
        requireNonnegative(second, "second");
        if (first == 0 || second == 0) {
            return OptionalLong.of(0);
        }
```

JAVA (continued 4/7)

```
        long reduced = first / gcd(first, second);
        if (reduced > Long.MAX_VALUE / second) {
            return OptionalLong.empty();
        }
        return OptionalLong.of(reduced * second);
    }

    public static long floorSquareRoot(long value) {
        requireNonnegative(value, "value");
        if (value < 2) {
            return value;
        }
        long low = 1;
        long high = Math.min(value / 2 + 1, 3_037_000_499L);
        long answer = 1;
        while (low <= high) {
            long mid = low + (high - low) / 2;
            if (mid <= value / mid) {
                answer = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return answer;
    }

    public static boolean isPerfectSquare(long value) {
        if (value < 0) {
            return false;
        }
        long root = floorSquareRoot(value);
        return root == 0
                ? value == 0
                : value / root == root && value % root == 0;
```

JAVA (continued 5/7)

```
    }

    public static boolean isPowerOfTwo(long value) {
        return value > 0 && (value & (value - 1)) == 0;
    }

    public static OptionalLong safePower(long base, int exponent) {
        if (exponent < 0) {
            throw new IllegalArgumentException(
                    "exponent must be nonnegative");
        }
        long result = 1;
        long factor = base;
        int remaining = exponent;

        while (remaining > 0) {
            if ((remaining & 1) != 0) {
                OptionalLong product = safeMultiply(result, factor);
                if (product.isEmpty()) {
                    return OptionalLong.empty();
                }
                result = product.getAsLong();
            }
            remaining >>>= 1;
            if (remaining > 0) {
                OptionalLong square = safeMultiply(factor, factor);
                if (square.isEmpty()) {
                    return OptionalLong.empty();
                }
                factor = square.getAsLong();
            }
        }
        return OptionalLong.of(result);
    }
```

JAVA (continued 6/7)

```
    public static long normalizeModulo(long value, long modulus) {
        if (modulus <= 0) {
            throw new IllegalArgumentException(
                    "modulus must be positive");
        }
        return Math.floorMod(value, modulus);
    }

    public static boolean isPerfectNumber(long value) {
        if (value <= 1) {
            return false;
        }
        long sum = 1;
        for (long divisor = 2;
                divisor <= value / divisor;
                divisor++) {
            if (value % divisor == 0) {
                long paired = value / divisor;
                if (sum > value - divisor) {
                    return false;
                }
                sum += divisor;
                if (paired != divisor) {
                    if (sum > value - paired) {
                        return false;
                    }
                    sum += paired;
                }
            }
        }
        return sum == value;
    }

    private static OptionalLong safeMultiply(long first, long second) {
        try {
```

JAVA (continued 7/7)

```
            return OptionalLong.of(Math.multiplyExact(first, second));
        } catch (ArithmeticException overflow) {
            return OptionalLong.empty();
        }
    }

    private static void requirePositive(long value) {
        if (value <= 0) {
            throw new IllegalArgumentException(
                    "value must be positive");
        }
    }

    private static void requireNonnegative(
            long value, String parameterName) {
        if (value < 0) {
            throw new IllegalArgumentException(
                    parameterName + " must be nonnegative");
        }
    }

    public static void main(String[] args) {
        System.out.println(isPrime(29));             // true
        System.out.println(factors(36));             // nine factors
        System.out.println(primeFactors(84));        // 2^2, 3, 7
        System.out.println(gcd(48, 18));             // 6
        System.out.println(safeLcm(21, 6));          // 42
        System.out.println(floorSquareRoot(27));     // 5
        System.out.println(isPerfectSquare(81));     // true
        System.out.println(isPowerOfTwo(1_024));     // true
        System.out.println(safePower(3, 5));         // 243
        System.out.println(normalizeModulo(-13, 5)); // 2
        System.out.println(isPerfectNumber(28));     // true
    }
}
```

The numeric-type choices are deliberate. Factor search uses long and a division boundary. LCM divides before multiplication. Fast exponentiation reports overflow instead of wrapping. The factor-count result fits comfortably in int for any positive long input, while factors themselves remain long.

WHY IT WORKS | 14.10 Large-number-string invariant

When a decimal value may contain thousands or millions of digits, it cannot be parsed into long. The solution should carry only the state needed by the question:

- a remainder for divisibility or modulo;
- a carry for addition;
- a canonical length and lexicographic comparison for ordering.

This is streaming reasoning. The full mathematical integer never needs to exist in memory as a primitive.

Pattern 11: Divisibility by 9 for an arbitrarily large number

Recognition signal: The input is a decimal string too large for primitive parsing, and only divisibility by nine matters.

Brute force: Parse with long and use value % 9, which fails outside the long range.

Better approach: Sum all digits, then test sum % 9.

Optimal interview approach: Maintain the sum modulo nine while scanning so the state remains small for arbitrarily long input.

Java solution: LargeNumberPatterns.isDivisibleBy9.

Complexity: O(d) time and O(1) extra space.

Dry run: "729" carries remainders 7, 0, 0 and is divisible by nine.

Edge cases: "0" and strings of zeros are divisible by nine. Invalid characters are rejected.

Common mistake: Parsing first, or allowing an int digit sum to overflow for an unbounded string.

Follow-up: Generalize the scan to compute a remainder modulo any positive int.

Pattern 12: Divisibility by 11 for an arbitrarily large number

Recognition signal: A decimal string must be tested with the alternating-sum rule.

Brute force: Parse the entire value.

Better approach: Build separate sums for alternating positions and compare their difference modulo eleven.

Optimal interview approach: Carry the alternating signed remainder modulo eleven in one scan.

Java solution: LargeNumberPatterns.isDivisibleBy11.

Complexity: O(d) time and O(1) extra space.

Dry run: "121" has alternating sum 1 - 2 + 1 = 0, so it is divisible by eleven.

Edge cases: Leading zeros do not change the value. Starting with either alternating sign gives a result that differs only by sign, so divisibility is unchanged.

Common mistake: Applying the digit rule without validating that every character is decimal.

Follow-up: Return the actual remainder of the huge decimal string modulo eleven.

Pattern 23: Modulo of a large numeric string

Recognition signal: A decimal string is too large to parse, but only value % modulus is needed.

Brute force: Use BigInteger or attempt primitive parsing.

Better and optimal approach: Apply Horner accumulation to the remainder:

```text
remainder = (remainder * 10 + digit) % modulus
```

Java solution: LargeNumberPatterns.modulo.

Complexity: O(d) time and O(1) space.

Dry run: "1234" modulo 7 carries 1, 5, 4, 2.

Edge cases: Modulus must be positive. This implementation uses an int modulus, so remainder * 10 + digit fits safely in long.

Common mistake: Storing the growing prefix rather than only its remainder.

Follow-up: Process characters from a Reader so the full string is not retained.

Pattern 24: Add two numeric strings

Recognition signal: Two nonnegative decimal values may exceed every primitive type.

Brute force: Parse both values and add them.

Better and optimal approach: Walk from right to left, add two digits and a carry, append the result digit, and reverse once.

Java solution: LargeNumberPatterns.add.

Complexity: O(max(a, b)) time and O(max(a, b)) output space.

Dry run: "999" + "1" produces digits 0, 0, 0 with carries, then a final 1, yielding "1000".

Edge cases: Different lengths, leading zeros, zero plus zero, and a final carry.

Common mistake: Forgetting to include the remaining carry after both inputs are exhausted.

Follow-up: Add signed numeric strings with canonical output.

Pattern 25: Compare two numeric strings

Recognition signal: Order two nonnegative decimal values without parsing them.

Brute force: Parse into long.

Better approach: Remove leading zeros, compare lengths, then compare characters.

Optimal interview approach: The better approach is already optimal because every relevant digit may need inspection.

Java solution: LargeNumberPatterns.compare.

Complexity: O(a + b) time to validate and canonicalize, and O(a + b) space in this clear string-producing implementation. An index-only variant uses O(1) extra space.

Dry run: "00098" and "101" canonicalize to "98" and "101"; shorter length means the first is smaller.

Edge cases: All-zero strings canonicalize to "0".

Common mistake: Lexicographically comparing raw strings before handling length and leading zeros.

Follow-up: Compare signed numeric strings while treating "-0" as zero.

14.11 Compiling large-number-string implementation

JAVA (continued 1/3)

```java
public final class LargeNumberPatterns {
    private LargeNumberPatterns() {
    }

    public static boolean isDivisibleBy9(String digits) {
        requireDigits(digits);
        int remainder = 0;
        for (int i = 0; i < digits.length(); i++) {
            int digit = digits.charAt(i) - '0';
            remainder = (remainder + digit) % 9;
        }
        return remainder == 0;
    }

    public static boolean isDivisibleBy11(String digits) {
        requireDigits(digits);
        int alternating = 0;
        int sign = 1;
        for (int i = 0; i < digits.length(); i++) {
            int digit = digits.charAt(i) - '0';
            alternating = (alternating + sign * digit) % 11;
            sign = -sign;
        }
        return alternating == 0;
    }

    public static int modulo(String digits, int modulus) {
        requireDigits(digits);
        if (modulus <= 0) {
            throw new IllegalArgumentException(
                    "modulus must be positive");
        }
        long remainder = 0;
        for (int i = 0; i < digits.length(); i++) {
            int digit = digits.charAt(i) - '0';
```

JAVA (continued 2/3)

```
            remainder = (remainder * 10 + digit) % modulus;
        }
        return (int) remainder;
    }

    public static String add(String first, String second) {
        requireDigits(first);
        requireDigits(second);
        int left = first.length() - 1;
        int right = second.length() - 1;
        int carry = 0;
        StringBuilder reversed = new StringBuilder(
                Math.max(first.length(), second.length()) + 1);

        while (left >= 0 || right >= 0 || carry != 0) {
            int firstDigit = left >= 0
                    ? first.charAt(left--) - '0'
                    : 0;
            int secondDigit = right >= 0
                    ? second.charAt(right--) - '0'
                    : 0;
            int sum = firstDigit + secondDigit + carry;
            reversed.append((char) ('0' + sum % 10));
            carry = sum / 10;
        }
        return canonical(reversed.reverse().toString());
    }

    public static int compare(String first, String second) {
        String left = canonical(first);
        String right = canonical(second);
        if (left.length() != right.length()) {
            return Integer.compare(left.length(), right.length());
        }
        return left.compareTo(right);
```

JAVA (continued 3/3)

```
    }

    public static String canonical(String digits) {
        requireDigits(digits);
        int firstNonzero = 0;
        while (firstNonzero < digits.length() - 1
                && digits.charAt(firstNonzero) == '0') {
            firstNonzero++;
        }
        return digits.substring(firstNonzero);
    }

    private static void requireDigits(String digits) {
        if (digits == null || digits.isEmpty()) {
            throw new IllegalArgumentException(
                    "digits must not be empty");
        }
        for (int i = 0; i < digits.length(); i++) {
            char current = digits.charAt(i);
            if (current < '0' || current > '9') {
                throw new IllegalArgumentException(
                        "invalid decimal digit at index " + i);
            }
        }
    }

    public static void main(String[] args) {
        System.out.println(isDivisibleBy9("729"));       // true
        System.out.println(isDivisibleBy11("121"));     // true
        System.out.println(modulo("1234", 7));          // 2
        System.out.println(add("000999", "1"));         // 1000
        System.out.println(compare("00098", "101"));    // negative
    }
}
```

14.12 Overflow-aware patterns

Overflow safety is not a cleanup step. It is part of the algorithm's contract. A wrapped result cannot reliably reveal whether overflow occurred because the same bit pattern may also be a legitimate result from different operands.

Pattern 26: Safe multiplication

Recognition signal: Two long operands must be multiplied only if the exact mathematical result fits.

Brute force: Multiply and inspect whether the result "looks wrong." This is not reliable.

Better approach: Write sign-sensitive division guards, carefully handling Long.MIN_VALUE and -1.

Optimal Java approach: Use Math.multiplyExact and convert ArithmeticException into the method's documented result type.

Java solution: OverflowPatterns.safeMultiply.

Complexity: O(1) time and O(1) space.

Dry run: Long.MAX_VALUE * 2 triggers overflow and returns OptionalLong.empty.

Edge cases: Zero, one, negative operands, Long.MIN_VALUE * -1, and two negative values.

Common mistake: Checking result / first == second after multiplication; overflow and zero cases make this fragile.

Follow-up: Implement a saturating multiplication policy instead of an optional result.

Pattern 27: Overflow-safe binary-search midpoint

Recognition signal: A midpoint is computed between two valid nonnegative indices.

Brute force: (left + right) / 2, whose addition can overflow.

Better and optimal approach for indices: left + (right - left) / 2 after validating 0 <= left <= right. The difference then fits int.

Java solution: OverflowPatterns.safeMidpoint.

Complexity: O(1) time and O(1) space.

Dry run: left = 1,500,000,000 and right = 2,000,000,000 produce difference 500,000,000 and midpoint 1,750,000,000.

Edge cases: Equal bounds and invalid negative or reversed bounds.

Common mistake: Reusing this proof for arbitrary signed endpoints where right - left itself may overflow.

Follow-up: Compute the average of arbitrary signed long values without overflow and define rounding for negative odd sums.

Related pattern: Safe LCM

Pattern 18 is also an overflow problem. Dividing by GCD before multiplying reduces the intermediate, but it does not prove that the remaining product fits. The final guard is still required.

Related pattern: Safe integer reversal

Pattern 3 uses a long accumulator for an int input. If the input and output were both long, a wider primitive would not exist; the loop would need a pre-update bound check or Math.multiplyExact and Math.addExact.

Extension: Comparator overflow

Sorting by subtraction is unsafe:

JAVA

```
// Wrong: subtraction can overflow and reverse the ordering.
values.sort((first, second) -> first - second);
```

Use the comparison API:

JAVA

```
values.sort(Integer::compare);
```

Recognition signal: A comparator orders fixed-width numbers.

Complexity: O(1) per comparison.

Edge cases: Integer.MIN_VALUE compared with a positive value.

Follow-up: Chain multiple keys with Comparator.comparingInt and thenComparing.

14.13 Compiling overflow implementation

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.OptionalLong;

public final class OverflowPatterns {
    private OverflowPatterns() {
    }

    public static OptionalLong safeMultiply(
            long first, long second) {
        try {
            return OptionalLong.of(
                    Math.multiplyExact(first, second));
        } catch (ArithmeticException overflow) {
            return OptionalLong.empty();
        }
    }

    public static int safeMidpoint(int left, int right) {
        if (left < 0 || left > right) {
            throw new IllegalArgumentException(
                    "require 0 <= left <= right");
        }
```

JAVA (continued 2/2)

```
        return left + (right - left) / 2;
    }

    public static List<Integer> sortedCopy(
            List<Integer> input) {
        List<Integer> result = new ArrayList<>(input);
        result.sort(Comparator.naturalOrder());
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        System.out.println(safeMultiply(12, 13)); // 156
        System.out.println(
                safeMultiply(Long.MAX_VALUE, 2)); // empty
        System.out.println(
                safeMidpoint(1_500_000_000, 2_000_000_000));
        System.out.println(
                sortedCopy(List.of(
                        Integer.MAX_VALUE,
                        0,
                        Integer.MIN_VALUE)));
    }
}
```

The exception in safeMultiply does not escape; it is translated into the explicit OptionalLong contract. In production code, an API may instead propagate ArithmeticException, return a domain result object, use BigInteger, or reject the operation before it begins.

RECAP | 14.14 Pattern-recognition summary

Problem wording	Carry this state	Usually avoid
"Each decimal digit"	quotient and remainder by 10	String parsing unless text is the real input
"Value in base b"	multiply-add or quotient/remainder by b	Math.pow for exact conversion
"Factors or prime"	divisor through n / divisor	scanning through n
"Cycles align"	GCD, divide, checked multiply	a * b before division
"Huge decimal string modulo m"	current remainder only	primitive parsing
"Add huge numbers"	indices and carry	converting to double
"Compare huge numbers"	canonical length then text	raw lexicographic order
"Exact fixed-width operation"	precondition or exact API	checking after wraparound
"Circular nonnegative index"	floor modulus	assuming % is nonnegative
"Positive power of two"	one-set-bit invariant	accepting zero

14.15 Interview communication template

For a numeric interview problem, a concise high-quality explanation sounds like this:

1. "I will define the input domain first: nonnegative decimal text, leading zeros allowed, invalid characters rejected."
2. "The entire value cannot fit in a primitive, but I only need its remainder."
3. "For each digit, the next prefix is oldPrefix * 10 + digit, so the next remainder is (oldRemainder * 10 + digit) % modulus."
4. "Because modulus is a positive int, the intermediate fits in long."
5. "The scan is O(d) time and O(1) extra space."
6. "I will test zero, leading zeros, invalid input, and the largest allowed modulus."

This explanation exposes the invariant, numeric safety, complexity, and tests before code.

WATCH OUT | 14.16 Common mistakes across the catalog

- Leaving the input domain implicit.
- Treating zero as having no digits.
- Applying Math.abs before promoting Integer.MIN_VALUE.
- Parsing a large numeric string before applying a divisibility rule.
- Using Math.pow for an exact integer conversion.
- Checking overflow only after a fixed-width operation wraps.
- Multiplying before dividing in LCM.
- Writing divisor * divisor <= n near a type boundary.
- Forgetting to validate a digit against the selected base.
- Returning an empty representation for zero.
- Duplicating the square-root factor.
- Treating one as prime.
- Omitting the positive guard in a power-of-two test.
- Using raw lexicographic order for decimal strings of different lengths.
- Forgetting a final addition carry.
- Assuming Java's remainder is always nonnegative.
- Sorting integers by subtraction.
- Stating O(log n) without naming what shrinks geometrically.

14.17 Practice set

Solutions are intentionally separated into the delayed notes.

Quick check

1. Why does countDigits need a special interpretation for zero?
2. What loop invariant makes Horner base parsing correct?
3. Why does a factor search stop at divisor <= n / divisor?
4. What state is sufficient to compute a huge decimal string modulo m?
5. Why is a / gcd(a, b) * b safer than a * b / gcd(a, b)?

Coding practice

1. **Foundation:** Return the sum and product of the digits of an int in one traversal.
2. **Foundation:** Validate a base-8 string.
3. **Foundation:** Return all factor pairs of a positive long.
4. **Interview Core:** Parse a nonnegative base-b string to int with exact overflow detection.
5. **Interview Core:** Add one to a nonnegative numeric string in place in a char array when capacity permits.
6. **Interview Core:** Determine whether a huge decimal string is divisible by both 9 and 11 in one scan.
7. **Interview Core:** Return the next power of two in OptionalLong.
8. **SDE-2 Follow-up:** Compute a huge decimal string modulo several positive int moduli in one pass.

Debugging tasks

1. Explain every failure in this prime loop:

```java
static boolean isPrime(int value) {
    for (int divisor = 2; divisor * divisor < value; divisor++) {
        if (value % divisor == 0) {
            return false;
        }
    }
    return true;
}
```

2. Explain why this comparator violates its contract for some inputs:

```java
values.sort((first, second) -> first - second);
```

3 Find the overflow path:

```java
static long lcm(long first, long second) {
    return first * second / gcd(first, second);
}
```

Interview extensions

1 Modify baseToLong to accept an optional leading sign and still parse Long.MIN_VALUE exactly.

2 Add signed large-number strings, canonicalizing "-0" to "0".

3 Compute (a * b) % modulus when a and b are nonnegative long values and their product may overflow.

4 Design a streaming API that computes divisibility while reading a very large file of digits.

14.18 Delayed solution notes

Quick-check answers

1 Decimal zero is represented by the single digit 0, so its digit count is one.

2 Before processing a character, result equals the numeric value of the processed prefix. Multiplying by the base shifts that prefix one position before the next digit is added.

3 Every factor above sqrt(n) is paired with a factor below sqrt(n), so no new pair begins beyond the boundary. Division expresses the boundary without square overflow.

4 The current prefix remainder is sufficient because congruent prefixes remain congruent after multiplying by ten and adding the same digit.

5 Dividing first reduces the factor before multiplication. A final bound check is still necessary.

Coding guidance

1. Promote to the input's safe magnitude type once, then update both aggregates from the same digit. The product of digits for an int fits in long.
2. Map only ASCII `0` through `9` and letters explicitly, then require a value from zero through seven for every character. `Character.digit` is convenient only when a broader Unicode-digit contract is intentional.
3. Emit divisor and n / divisor for each exact division; avoid duplicating the square root.
4. Use the same bound result > (Integer.MAX_VALUE - digit) / base before each update.
5. Start from the rightmost character, propagate carry, and report whether a new leading character is required.
6. Carry two remainders: sum modulo nine and alternating sum modulo eleven.
7. Reject nonpositive input or define its contract, return the input if already a power of two, and check current > Long.MAX_VALUE / 2 before doubling.
8. Store one remainder per requested modulus. Time is O(d * m) for d digits and m moduli; space is O(m).

Debugging resolutions

The prime method must reject values below two. Its boundary must include an exact square root, and divisor * divisor can overflow. A safe condition is divisor <= value / divisor.

Comparator subtraction can overflow, reverse the sign, and violate transitivity. Use Integer.compare(first, second) or Comparator.naturalOrder().

LCM multiplication may overflow before division, and gcd(0, 0) can create division by zero depending on the implementation. Handle zero, divide by GCD first, and check the remaining multiplication.

Extension strategies

- A signed base parser can accumulate negatively, as Java's parsing implementations commonly do, because the negative range contains one extra magnitude. Validate the sign and compare against a negative limit before each update.
- Signed string addition first canonicalizes signs and magnitudes. Equal signs add magnitudes; different signs subtract the smaller magnitude from the larger.
- Overflow-safe modular multiplication can use repeated doubling with modular addition, BigInteger, or a carefully proven unsigned technique. State constraints before choosing.
- A streaming digit API carries remainder state across chunks and records an absolute character offset for validation errors.

RECAP | 14.19 Chapter summary

- Digit traversal removes one decimal digit per iteration.
- Horner accumulation is the central base-to-decimal invariant.
- Repeated division produces digits from least significant to most significant.
- Factor pairs reduce divisor search to O(sqrt(n)).
- Euclid's algorithm computes GCD in logarithmic time.
- Safe LCM divides before a checked multiply.
- Large-number strings often need only remainder, carry, or canonical length state.
- Exact arithmetic must detect overflow before trusting the result.
- Java exact-arithmetic and comparison APIs encode difficult boundary behavior clearly.

TRY IT | 14.20 Revision checklist

[ ] I can identify all 30 mandatory patterns in the coverage map.
[ ] I can implement digit traversal for zero and Integer.MIN_VALUE.
[ ] I can parse and format bases 2 through 36.
[ ] I can explain the square-root factor boundary without multiplying.
[ ] I can derive GCD and overflow-safe LCM.
[ ] I can process a numeric string that does not fit in long.
[ ] I can add and compare numeric strings with leading zeros.
[ ] I can use Math.multiplyExact and a safe midpoint.
[ ] I can normalize negative modulo with Math.floorMod.
[ ] I can state complexity, numeric safety, and edge cases before coding.

SDE-2 CORE CHAPTER

Chapter 2 - Chapter 14A: Fifty-Two Implementation Reference

Part A taught the ideas. Chapter 14 turned thirty high-frequency ideas into reusable patterns. This chapter closes the implementation gap: it indexes all fifty-two required solutions, explains the newly added techniques, and points to the compiling Java 21 companion.

The canonical implementations are in `code/NumberSystemsAlgorithms.java`; the boundary suite is in `code/NumberSystemsAlgorithmsTest.java`. Do not memorize all fifty-two methods. Memorize the small invariants they share: traverse a digit, accumulate a base, pair factors, reduce with GCD, normalize a remainder, halve an exponent, and check before an operation can overflow.

14A.1 Reading route

Use this chapter in three passes:

1. **Foundation:** implement digit statistics, factorial, base validation, and factor sums.
2. **Interview Core:** implement strict reversal, numeric-string arithmetic, sieve, array GCD/LCM, and bit length.
3. **SDE-2 Follow-up:** implement factorial counting formulas and modular inverse, then explain when each contract fails.

If an invariant is unfamiliar, return to the prerequisite chapter named in the index. If the invariant is clear but the code is slow or fragile, stay here and compare boundary policies.

14A.2 Complete implementation index

The four smaller tables are intentional: they remain readable on paper and prevent one oversized catalog from splitting unpredictably across pages.

Implementations 1-13

#	Required implementation	Compiling method	Level
1	Count digits	`countDigits`	Foundation
2	Sum of digits	`sumDigits`	Foundation
3	Product of digits	`productDigits`	Foundation
4	Minimum digit	`minimumDigit`	Foundation
5	Maximum digit	`maximumDigit`	Foundation
6	Count occurrence of a digit	`countDigitOccurrences`	Foundation
7	Reverse integer	`reverseInt`	Interview Core
8	Strict overflow-safe reverse	`reverseIntStrict`	Interview Core
9	Palindrome number	`isPalindromeNumber`	Interview Core
10	Armstrong number	`isArmstrongNumber`	Foundation
11	Strong number	`isStrongNumber`	Foundation
12	Perfect number	`isPerfectNumber`	Optional Advanced
13	Factorial	`factorialExact`	Foundation

Implementations 14-26

#	Required implementation	Compiling method	Level
14	Decimal to binary	`decimalToBinary`	Foundation
15	Binary to decimal	`binaryStringToLong`	Interview Core
16	Decimal to generic base	`longToBase`	Interview Core
17	Generic base to decimal	`baseToLong`	Interview Core
18	Validate a number in a base	`isValidNumberInBase`	Interview Core
19	Hexadecimal to decimal	`hexadecimalToLong`	Foundation
20	Decimal to hexadecimal	`decimalToHexadecimal`	Foundation
21	Compare huge numeric strings	`compareNumericStrings`	Interview Core
22	Add huge numeric strings	`addNumericStrings`	Interview Core
23	Subtract huge numeric strings	`subtractNumericStrings`	SDE-2 Follow-up
24	Huge-number modulo	`largeNumberModulo`	Interview Core
25	Huge-number divisibility by 9	`isDivisibleBy9`	Interview Core
26	Huge-number divisibility by 11	`isDivisibleBy11`	Interview Core

Implementations 27-39

#	Required implementation	Compiling method	Level
27	List factors	`listFactors`	Interview Core
28	Count factors	`countFactors`	Interview Core
29	Sum factors	`sumFactors`	Interview Core
30	Prime check	`isPrime`	Interview Core
31	Prime factorization	`primeFactorization`	Interview Core
32	Sieve of Eratosthenes	`sievePrimes`	Interview Core
33	GCD	`gcd`, `gcdMagnitude`	Interview Core
34	LCM	`lcm`, `lcmMagnitude`	Interview Core
35	GCD of an array	`gcdOfArray`	Interview Core
36	LCM of an array	`lcmOfArray`	SDE-2 Follow-up
37	Normalize modulo	`normalizeModulo`	Interview Core
38	Modular addition	`addModulo`	Interview Core
39	Modular subtraction	`subtractModulo`	Interview Core

Implementations 40-52

#	Required implementation	Compiling method	Level
40	Modular multiplication	`multiplyModulo`	SDE-2 Follow-up
41	Fast exponentiation	`fastPowerExact`	Interview Core
42	Modular exponentiation	`powerModulo`	Interview Core
43	Perfect-square check	`isPerfectSquare`	Interview Core
44	Integer square root	`integerSquareRoot`	Interview Core
45	Power-of-two check	`isPowerOfTwo`	Foundation
46	Count number of bits	`countBits`	Interview Core
47	Trailing zeros in factorial	`trailingZerosInFactorial`	Interview Core
48	Number of digits in factorial	`digitsInFactorial`	SDE-2 Follow-up
49	Safe binary-search midpoint	`safeIndexMidpoint`, `signedMidpoint`	Interview Core
50	Overflow-safe comparator	`compareInts`	Interview Core
51	Safe integer multiplication	`safeMultiply`	Interview Core
52	Modular inverse	`modularInverse`	SDE-2 Follow-up

Numeric-string multiplication by one digit is also included as `multiplyNumericStringByDigit`; it supports implementation 23 and is a useful bridge to full grade-school multiplication.

14A.3 Digit statistics and strict reversal

Problem statement. Find minimum/maximum digits, count a target digit, or reverse an `int` under a strict no-wider-accumulator policy.

Why interviewers ask it. These tasks expose whether you can separate the digit-traversal invariant from zero, sign, and overflow policies.

Prerequisites. Chapter 2's `% 10` and `/ 10` loop and Chapter 7's pre-operation boundary checks.

Natural approach and limitation. Converting to text is simple but changes the problem to character processing. A `long` accumulator makes reversal safe for `int`, but it does not demonstrate how to protect a fixed-width multiply-add.

Optimal interview approach. Keep the working value nonpositive when a magnitude might be `MIN_VALUE`. For strict reversal, reject the next digit before evaluating `reversed * 10 + digit`.

JAVA

```java
static int minimumDigit(long value) {
    long remaining = value > 0 ? -value : value;
    int minimum = 9;
    do {
        minimum = Math.min(minimum, (int) -(remaining % 10));
        remaining /= 10;
    } while (remaining != 0);
    return minimum;
}

static int countDigitOccurrences(long value, int target) {
    if (target < 0 || target > 9) throw new IllegalArgumentException();
    long remaining = value > 0 ? -value : value;
    int count = 0;
    do {
        if ((int) -(remaining % 10) == target) count++;
        remaining /= 10;
    } while (remaining != 0);
    return count;
}
```

For strict positive overflow, `reversed > MAX_VALUE / 10` is already too large. At equality, the incoming digit must not exceed 7. The mirrored negative boundary permits -8 because `Integer.MIN_VALUE` ends in 8.

State	remaining	incoming digit	decision
Safe prefix	214,748,364	1	append; result fits
Positive boundary	214,748,364	8	reject; exceeds MAX_VALUE
Negative boundary	-214,748,364	-8	append; equals MIN_VALUE

Complexity. O(d) time and O(1) auxiliary space for d decimal digits.

Edge cases. Zero is one digit; signs are ignored for statistics; trailing zeros disappear during numeric reversal; `MIN_VALUE` must not be passed through same-type `abs`.

Common mistakes. A `while` loop skips zero, initialization of minimum to zero makes every positive input wrong, and checking overflow after the `int` operation is too late.

Follow-ups. Return all digit statistics in one traversal; reconstruct only half a palindrome; generalize the loop to another base.

14A.4 Factorial and its counting follow-ups

Problem statement. Compute `n!` when it fits, count trailing zeros of `n!`, or find its decimal digit count without constructing the value.

Why interviewers ask it. The three variants distinguish direct computation, factor counting, and logarithmic transformation.

Prerequisites. Multiplication, integer division, logarithm intuition, and explicit range contracts.

The natural factorial loop is optimal for a small exact result. `20!` fits in `long`; `21!` does not. The contract therefore accepts only 0 through 20 and begins at the identity `0! = 1`.

JAVA

```java
static long factorialExact(int n) {
    if (n < 0 || n > 20) throw new IllegalArgumentException();
    long result = 1;
    for (int factor = 2; factor <= n; factor++) {
        result = Math.multiplyExact(result, factor);
    }
    return result;
}

static long trailingZeros(int n) {
    if (n < 0) throw new IllegalArgumentException();
    long zeros = 0;
    for (long divisor = 5; divisor <= n; divisor *= 5) {
        zeros += n / divisor;
        if (divisor > n / 5) break;
    }
    return zeros;
}
```

For `100!`, multiples of 5 contribute 20 factors of five and multiples of 25 contribute four additional factors, so the answer is 24. Factors of two are more plentiful and do not limit trailing zeros.

For digit count, use `floor(sum(log10(k), k=2..n)) + 1`. This is O(n) time and O(1) space. It is a counting technique, not a substitute when exact factorial digits are required.

Edge cases. `0!` and `1!` both have one digit and zero trailing zeros. Negative factorial is outside the integer interview contract.

Common mistakes. Returning zero for `0!`, using `int`, counting only `n / 5`, or constructing a huge `BigInteger` merely to call `toString().length()`.

Follow-ups. Compute the last nonzero digit, use a prefix table for repeated small queries, or discuss Stirling's approximation as optional theory.

14A.5 Base validation and huge-string arithmetic

Problem statement. Validate a signed representation in base 2-36, subtract two signed decimal strings, or multiply a signed decimal string by one digit.

Why interviewers ask it. These tasks test parsing contracts, carry/borrow state, leading-zero normalization, and the ability to avoid unsupported primitive parsing.

Prerequisites. Chapters 5 and 8.

Validation must happen before accumulation. An optional sign is allowed only at index zero and must be followed by at least one digit. `Character.digit(character, base)` is convenient for general Java text; the companion intentionally uses an explicit ASCII contract so accepted symbols are predictable.

For subtraction, normalize signs and magnitudes first. Equal signs become magnitude subtraction; different signs become magnitude addition. The companion reuses signed addition by negating the second canonical operand, which prevents a second, inconsistent sign engine.

JAVA

```java
static String multiplyByDigit(String digits, int multiplier) {
    if (multiplier < 0 || multiplier > 9) throw new IllegalArgumentException();
    if (digits.equals("0") || multiplier == 0) return "0";
    int carry = 0;
    StringBuilder reversed = new StringBuilder(digits.length() + 1);
    for (int index = digits.length() - 1; index >= 0; index--) {
        int product = (digits.charAt(index) - '0') * multiplier + carry;
        reversed.append((char) ('0' + product % 10));
        carry = product / 10;
    }
    if (carry != 0) reversed.append(carry);
    return reversed.reverse().toString();
}
```

Step for `999 x 7`	digit product + carry	output digit	next carry
rightmost 9	63	3	6
middle 9	69	9	6
leftmost 9	69	9	6
final carry	-	6	0

The result is `6993`. Time is O(n) and output space is O(n). Per-position arithmetic is bounded because `9 * 9 + 8` is at most 89.

Edge cases. Empty or sign-only text is invalid; canonical zero has no negative sign; leading zeros are removed from numeric results; subtraction must define which sign wins.

Common mistakes. Parsing to `long`, comparing unnormalized lengths, dropping the final carry, retaining `-0`, or allowing a digit equal to the base.

Follow-ups. Multiply two numeric strings, convert a huge binary string with `BigInteger`, or stream several remainders in one pass.

14A.6 Factor sums, sieve, and array GCD/LCM

Problem statement. Sum every factor, preprocess all primes to a limit, or reduce GCD/LCM across an array.

Why interviewers ask it. These variations test whether a pairwise invariant can become enumeration, preprocessing, or reduction.

Prerequisites. Chapter 10's factor pairs and Euclidean algorithm.

Factor sum mirrors factor count: for each divisor through the square root, add the divisor and its partner, but add a square root once. Use `Math.addExact` when the return contract is `long`.

For the sieve, mark `prime * prime`, `prime * prime + prime`, and so on. The outer condition `prime <= limit / prime` proves that the starting product fits. Preprocessing costs O(n log log n) time and O(n) space; each later primality query is O(1).

Array GCD is a fold:

JAVA

```java
long result = 0;
for (long value : values) {
    result = gcd(result, value);
}
```

Zero is the identity because `gcd(0, x) = abs(x)`. Array LCM begins at one, but any zero makes the result zero. Each pair must divide by GCD before exact multiplication. Stop early after the result becomes zero.

Complexity. Factor sum is O(sqrt(n)); sieve is O(n log log n) time and O(n) space; array GCD is O(k log M) for k values bounded by magnitude M. Array LCM has the same number of reductions but may overflow much sooner.

Edge cases. Factor operations require positive input; sieve limits below two return an empty list; array contracts reject null and empty arrays; a GCD magnitude of 2^63 needs `BigInteger`.

Common mistakes. Double-counting the square root, starting sieve marks at `2 * p`, evaluating `p * p` before proving it fits, and multiplying before dividing in LCM.

Follow-ups. Prefix prime counts, divisor count from prime exponents, fraction reduction, or schedule alignment by array LCM.

14A.7 Bit length and modular inverse

Count bits

Problem statement. Return the number of binary positions required to represent a nonnegative integer. This means binary length, not population count.

For zero, this book returns one because textual binary zero is `0`. For positive `value`, repeated unsigned right shift takes O(log value) time. Java's `Long.numberOfLeadingZeros` gives the same answer in a constant number of machine-level operations:

JAVA

```java
static int countBits(long value) {
    if (value < 0) throw new IllegalArgumentException();
    return value == 0 ? 1 : Long.SIZE - Long.numberOfLeadingZeros(value);
}
```

Do not confuse bit length with `Long.bitCount`, which counts one-bits. Full bit-count patterns remain in the Bit Manipulation mini-book.

Modular inverse

Problem statement. Find `x` such that `(a * x) mod m = 1`.

Why interviewers ask it. It tests whether the candidate knows that division under modulo is not ordinary integer division and that an inverse exists only under a GCD condition.

Prerequisites. GCD, normalized modulo, and the identity produced by extended Euclid.

The natural search checks every `x` from 1 to `m - 1`, which is O(m). Extended Euclid finds coefficients `x` and `y` satisfying:

TEXT

```
a*x + m*y = gcd(a, m)
```

When the GCD is one, reducing `x` modulo `m` gives the inverse. When the GCD is not one, no inverse exists.

For `a = 3` and `m = 11`:

TEXT

```
1 = 4*3 - 1*11
therefore x = 4, and (3*4) mod 11 = 1
```

The companion uses iterative extended Euclid with `BigInteger` coefficients. This preserves correctness when intermediate coefficients exceed `long`, while returning a `long` inverse because the positive modulus itself is a `long`.

Complexity. O(log m) Euclidean iterations and O(log m)-sized coefficient arithmetic.

Edge cases. Require `m > 1`; normalize negative `a`; reject `gcd(a, m) != 1`; never assume a prime modulus unless the problem states it.

Common mistakes. Using integer division, applying Fermat's theorem to a composite modulus, returning a negative coefficient without normalization, or multiplying unrestricted `long` values before modulo.

Follow-ups. Inverse of every value under a prime modulus, fraction reduction before modular division, and safe modular multiplication for the final verification.

14A.8 Boundary matrix and recall

Input shape	Required response
`0` in a digit algorithm	Process one digit when the task counts or classifies digits
`MIN_VALUE`	Avoid same-type absolute value and negation
Huge numeric string	Validate and traverse; never parse the entire value to a primitive
Factor or prime loop	Use `factor <= value / factor`
Array LCM	Handle zero, divide first, then multiply exactly
Negative remainder	Normalize with a positive modulus
No modular inverse	Report the GCD failure; do not invent a quotient
Floating estimate	Verify with integer arithmetic when the answer is discrete

Quick check

1. Why does strict reversal check before the multiply-add?
2. Why do trailing zeros count factors of five rather than ten?
3. Why can a sieve start marking at `p * p`?
4. What identity makes zero the starting value for an array GCD?
5. What condition decides whether a modular inverse exists?

Guided practice

1. **Foundation:** Trace minimum, maximum, and zero count for `-900507`.
2. **Interview Core:** Trace `1000000 - 1` with right-to-left borrows.
3. **Interview Core:** Draw the sieve states through 40.
4. **SDE-2 Follow-up:** Derive the inverse of 17 modulo 43 with extended Euclid.

Independent coding

1. **Foundation:** Return sum, product, minimum, maximum, and digit count in one immutable result.
2. **Interview Core:** Implement strict reversal without `long` and test both int boundaries.
3. **Interview Core:** Implement signed numeric-string subtraction with canonical zero.
4. **Interview Core:** Return prefix prime counts after one sieve.
5. **Interview Core:** Reduce the GCD of an array and stop early at one.
6. **SDE-2 Follow-up:** Reduce array LCM with exact overflow reporting.
7. **SDE-2 Follow-up:** Return the trailing-zero count for `Integer.MAX_VALUE` without overflow.
8. **Optional Advanced:** Return a modular inverse using only documented bounded `long` inputs.

Debugging task

The following code can overflow before it decides to stop:

```java
for (int prime = 2; prime * prime <= limit; prime++) {
    if (!composite[prime]) {
        for (int multiple = prime * prime;
                multiple <= limit;
                multiple += prime) {
            composite[multiple] = true;
        }
    }
}
```

Identify the unsafe expression, replace the outer condition with a division guard, and state why the inner starting product is then representable.

Interview follow-up

The modulus is close to `Long.MAX_VALUE`, and the interviewer asks you to verify `(value * inverse) mod modulus == 1`. Explain why ordinary multiplication is unsafe and route the verification through the overflow-free modular multiplication method.

RECAP | 14A.9 Chapter summary

The complete Number Systems stage now has compiling references for all fifty-two required implementations. The additions do not create twenty-two unrelated tricks. They reuse seven ideas: bounded digit traversal, pre-operation overflow checks, carry/borrow arithmetic, factor pairing, preprocessing, associative reduction, and Euclidean identities.

When revising, name the contract and invariant before the method. That is the difference between recalling code and being able to adapt it under an SDE-2 follow-up.

SDE-2 CORE CHAPTER

Chapter 3 - Java Interview Traps Related to Numbers

Many Java number questions are not mathematics questions. They test whether the candidate tracks compile-time types, promotions, boxing, fixed-width overflow, parsing contracts, and floating-point semantics. The dangerous answers are often plausible because they would be correct in mathematical arithmetic or in a different programming language.

The defense is a repeatable evaluation order:

1. Determine the compile-time type of each operand.
2. Apply unboxing and numeric promotion.
3. Evaluate the operation in that promoted type.
4. Account for overflow, truncation, or floating-point rounding.
5. Apply assignment conversion or an explicit cast last.

15.1 Learning objectives

After this chapter, you should be able to:

- choose equals rather than identity for boxed numeric values;
- explain the limited boxing identity guarantee without relying on cache accidents;
- identify hidden unboxing and null failures;
- predict integer division and remainder;
- compare floating-point results under an explicit policy;
- place casts or long operands before an operation;
- detect overflow that occurs before assignment;
- handle MIN_VALUE absolute-value traps;
- distinguish remainder from floor modulus;
- predict shift promotion and distance masking;
- validate parsing signs, zeros, digits, and range;
- avoid comparator subtraction overflow.

15.2 Boxed identity versus numeric equality

The operator == compares primitive numeric values when both operands are primitive. When both operands are references such as Integer, it compares object identity.

JAVA

```
Integer first = 1_000;
Integer second = 1_000;

System.out.println(first == second);      // do not rely on this
System.out.println(first.equals(second)); // true
```

Boxing selected constant values has an identity guarantee. In particular, boxed int constants from -128 through 127 are required to share identity in the situations described by the language specification. Implementations may cache additional values. Neither fact makes identity the right value-comparison operation.

JAVA

```
Integer smallFirst = 127;
Integer smallSecond = 127;
System.out.println(smallFirst == smallSecond); // true
```

The interview rule is simple:

- use == for primitive numeric comparison;
- use equals for same-wrapper value equality when null handling is explicit;
- use Objects.equals when either wrapper may be null;
- use a numeric conversion or comparator when wrapper types differ.

Wrapper equals methods are type-sensitive:

JAVA

```
Integer integer = 1;
Long longValue = 1L;
System.out.println(integer.equals(longValue)); // false
```

Both objects represent the mathematical value one, but Integer.equals requires another Integer.

Common candidate mistake

Explaining a large boxed comparison as "always false." Identity outside the required cache range is not a stable value contract. The correct conclusion is that the program should not depend on it.

15.3 Autoboxing, unboxing, and null

Autoboxing converts a primitive to its wrapper when the context requires a reference. Unboxing extracts a primitive when an arithmetic or primitive context requires it.

JAVA

```java
Integer boxed = 40;      // boxing
int result = boxed + 2;  // unboxing, then int addition
```

If boxed is null, unboxing throws NullPointerException:

JAVA

```java
Integer missing = null;
int value = missing; // NullPointerException
```

The exception occurs at the unboxing point, not when null was assigned.

Hidden unboxing can occur in:

- arithmetic;
- comparison with a primitive;
- switch expressions or statements;
- method invocation requiring a primitive;
- compound assignments;
- the conditional operator, depending on operand types.

Prefer primitives for required numeric values. If absence is meaningful, validate or model it before arithmetic:

JAVA

```java
static int requireValue(Integer value) {
    if (value == null) {
        throw new IllegalArgumentException("value is required");
    }
    return value;
}
```

Overload awareness

Overload selection considers primitive widening, boxing, and varargs under ordered rules. Do not guess from runtime wrapper identity. Determine the declared argument type and applicable methods at compile time.

15.4 Integer division

When both operands are integral, Java performs integral division and discards the fractional part. The result rounds toward zero.

JAVA

```
System.out.println(7 / 2);   // 3
System.out.println(-7 / 2);  // -3
System.out.println(7 / -2);  // -3
```

A later assignment to double does not recover the fraction:

JAVA

```
double wrong = 7 / 2;         // 3.0
double correct = 7.0 / 2;     // 3.5
double alsoCorrect = (double) 7 / 2;
```

At least one operand must become floating point before division.

Integer division by zero throws ArithmeticException. Floating-point division by zero follows IEEE 754 behavior and may produce infinity or NaN:

JAVA

```
System.out.println(1.0 / 0.0); // Infinity
System.out.println(0.0 / 0.0); // NaN
```

15.5 Floating-point precision

Most finite decimal fractions do not have an exact finite binary floating-point representation. Therefore:

JAVA

```
double computed = 0.1 + 0.2;
System.out.println(computed == 0.3); // false
```

This is not random error. Each operand is rounded to a representable binary value, and the arithmetic result is rounded again.

Choose a comparison policy

There is no universal epsilon. A useful policy depends on the scale, units, accumulated operations, and domain.

For many interview examples, combine absolute and relative tolerance:

JAVA

```java
static boolean nearlyEqual(
        double first,
        double second,
        double absoluteTolerance,
        double relativeTolerance) {
    if (!Double.isFinite(absoluteTolerance)
            || !Double.isFinite(relativeTolerance)
            || absoluteTolerance < 0
            || relativeTolerance < 0) {
        throw new IllegalArgumentException(
                "tolerances must be finite and nonnegative");
    }
    if (Double.isNaN(first) || Double.isNaN(second)) {
        return false;
    }
    if (first == second) {
        return true;
    }
    if (!Double.isFinite(first) || !Double.isFinite(second)) {
        return false;
    }
    double difference = Math.abs(first - second);
    double scale = Math.max(Math.abs(first), Math.abs(second));
    return difference <= Math.max(
            absoluteTolerance,
            relativeTolerance * scale);
}
```

The `first == second` check deliberately accepts equal infinities and both signed zeros. NaN is handled first because NaN is unequal to every value, including itself. After the exact-equality case, any remaining infinity is rejected; otherwise both the difference and relative threshold could become infinity and create a false match. Tolerances must themselves be finite and nonnegative.

For ordering, Double.compare provides a defined total ordering suitable for comparators. Approximate equality is generally not a valid comparator equivalence because it may violate transitivity.

Decimal business values

When exact decimal arithmetic is required, BigDecimal may be appropriate:

JAVA

```java
import java.math.BigDecimal;

BigDecimal tenth = new BigDecimal("0.1");
BigDecimal twoTenths = new BigDecimal("0.2");
System.out.println(tenth.add(twoTenths)); // 0.3
```

Construct from a decimal string or use BigDecimal.valueOf when starting from a double. new BigDecimal(0.1) preserves the exact binary double value, which is usually not the intended decimal 0.1.

15.6 Casting order

A cast changes the value and type at the point where it appears. Casting the result is different from casting an operand.

JAVA

```java
int completed = 3;
int total = 4;

double wrong = (double) (completed / total); // 0.0
double right = (double) completed / total;   // 0.75
```

In wrong, integer division happens inside the parentheses before the cast. In right, completed becomes double first, so binary numeric promotion makes the division floating point.

Narrowing casts can discard high bits:

JAVA

```java
int value = 130;
byte narrowed = (byte) value; // -126
```

The cast does not clamp to Byte.MAX_VALUE and does not throw.

15.7 Overflow before assignment

An expression is evaluated according to operand types, not the destination type.

JAVA

```java
long wrong = Integer.MAX_VALUE * 2;
long right = Integer.MAX_VALUE * 2L;
```

In wrong, both operands are int, so multiplication overflows in int and the wrapped result is widened to long. In right, 2L promotes the other operand and multiplication occurs in long.

The same issue appears with constants and variables:

JAVA

```java
long seconds = 30 * 24 * 60 * 60; // fits here, but fragile
long safer = 30L * 24 * 60 * 60;
```

Put the long operand early enough that every subsequent multiplication is long.

Exact APIs make the policy explicit:

```java
long product = Math.multiplyExact(first, second);
long sum = Math.addExact(first, second);
int incremented = Math.incrementExact(value);
```

They throw ArithmeticException when the exact mathematical result does not fit.

15.8 Math.abs and MIN_VALUE

Two's-complement signed types have one more negative value than positive value. Therefore:

```java
System.out.println(Math.abs(Integer.MIN_VALUE));
// -2147483648
```

No positive int can represent 2,147,483,648.

For an int magnitude, promote first:

```java
long magnitude = Math.abs((long) value);
```

For Long.MIN_VALUE, no wider primitive integer exists. Choose among:

- reject or special-case it;
- use BigInteger;
- use unsigned magnitude logic;
- structure the algorithm to avoid absolute value;
- accumulate in the negative range, as robust signed parsers do.

Negating MIN_VALUE has the same trap:

```java
int stillMinimum = -Integer.MIN_VALUE;
```

15.9 Remainder versus mathematical modulus

Java's % is remainder, and its sign follows the dividend:

```java
System.out.println(-13 % 5); // -3
System.out.println(13 % -5); // 3
```

For circular indices with positive modulus, use Math.floorMod:

```java
System.out.println(Math.floorMod(-13, 5)); // 2
```

The common normalization expression:

```java
((value % modulus) + modulus) % modulus
```

works under suitable bounds and a positive modulus, but Math.floorMod states the intent and handles fixed-width details clearly.

15.10 Shift behavior

Shift traps combine promotion, sign handling, overflow, and distance masking.

Operand promotion

byte, short, and char are promoted to int before shifting:

```java
byte one = 1;
int shifted = one << 8; // int 256
```

Width comes from the left operand

```java
long wrong = 1 << 40;  // int shift; distance becomes 8
long right = 1L << 40; // long shift
```

Distance is masked

- int uses distance & 31;
- long uses distance & 63.

```java
System.out.println(1 << 32);  // 1
System.out.println(1L << 64); // 1
```

Signed and unsigned right shift

```java
System.out.println(-1 >> 1);  // -1
System.out.println(-1 >>> 1); // 2147483647
```

Do not replace signed division with right shift without checking rounding for negative odd values.

15.11 Parsing contracts

Parsing can fail because of syntax or range:

```java
Integer.parseInt("+42");         // 42
Integer.parseInt("-42");         // -42
Integer.parseInt("00042");       // 42, decimal
Integer.parseInt(" 42 ");        // NumberFormatException
Integer.parseInt("2147483648");  // NumberFormatException
```

Integer.parseInt does not trim whitespace automatically. It accepts one leading plus or minus sign but not a sign by itself.

Leading zeros do not make parseInt(String) octal. Radix-sensitive APIs have separate contracts:

```java
Integer.parseInt("10", 2);  // 2
Integer.parseInt("FF", 16); // 255
```

Integer.decode recognizes prefixes such as 0x and # and has different leading-zero behavior. Do not substitute it for a clearly specified decimal parser.

Parsing unsigned text

Integer.parseUnsignedInt can parse a nonnegative 32-bit value that does not fit in positive int, but the returned int may have a negative signed interpretation. Use Integer.toUnsignedLong or unsigned comparison/formatting APIs when needed.

Exception policy

For trusted configuration, allowing NumberFormatException to identify invalid input may be fine. At a public boundary, validate length, signs, radix, and error reporting according to the API contract. Do not catch every exception and silently return zero; zero may be a valid input.

15.12 Leading signs and zeros

A robust parser must state:

- whether one leading + or - is accepted;
- whether the sign may be followed by separators or whitespace;
- whether leading zeros are allowed;
- whether "-0" canonicalizes to "0";
- whether a prefix such as 0x is accepted;
- what happens on a sign-only string;
- whether non-ASCII digits are accepted.

For interview code, a narrow explicit contract is better than accidental flexibility.

Leading zeros matter differently by task:

- numeric comparison should ignore them after validation;
- string identity should preserve them;
- fixed-width identifiers may treat them as significant;
- decimal parsing treats them as ordinary decimal zeros;
- Integer.decode can interpret prefixes under its own rules.

15.13 Character-to-digit conversion

Subtracting '0' is clear for validated ASCII decimal digits:

JAVA

```java
static int asciiDecimalDigit(char value) {
    if (value < '0' || value > '9') {
        throw new IllegalArgumentException("not an ASCII digit");
    }
    return value - '0';
}
```

Do not subtract '0' before validation. For hexadecimal and other radices, use Character.digit:

JAVA

```java
static int digitInBase(char value, int base) {
    int digit = Character.digit(value, base);
    if (digit < 0) {
        throw new IllegalArgumentException("digit not valid in base");
    }
    return digit;
}
```

Character.digit handles letter case and more Unicode characters than an ASCII-only contract. If the input must be ASCII, add an explicit character-range policy rather than assuming the method enforces one.

15.14 Comparator overflow

A comparator's sign communicates order. Subtraction can wrap and report the wrong sign:

JAVA

```java
// Unsafe
Comparator<Integer> wrong =
        (first, second) -> first - second;

// Safe
Comparator<Integer> right = Integer::compare;
```

Consider first = Integer.MIN_VALUE and second = 1. The mathematical difference is below Integer.MIN_VALUE, so int subtraction wraps positive. The unsafe comparator claims the minimum value is greater than one.

For long use Long.compare, for double use Double.compare, and for object keys use Comparator.comparingInt, comparingLong, or comparing.

Comparator correctness requires antisymmetry, transitivity, and consistent zero-equivalence. Approximate floating-point equality usually should not define comparator equality.

15.15 Additional promotion trap: compound assignment

Compound assignment includes an implicit cast to the left-hand type:

JAVA

```java
byte value = 127;
value += 1;
System.out.println(value); // -128
```

The expression behaves roughly like:

JAVA

```java
value = (byte) (value + 1);
```

By contrast, value = value + 1 does not compile without a cast because value + 1 is int.

Unary and binary numeric promotion also mean:

JAVA

```java
byte small = 10;
int negated = -small;
int added = small + small;
```

Both results are int.

15.16 Runnable trap laboratory

JAVA (continued 1/4)

```java
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Objects;

public final class JavaNumberTraps {
    private JavaNumberTraps() {
    }

    public static boolean nearlyEqual(
            double first,
            double second,
            double absoluteTolerance,
            double relativeTolerance) {
        if (!Double.isFinite(absoluteTolerance)
                || !Double.isFinite(relativeTolerance)
                || absoluteTolerance < 0
                || relativeTolerance < 0) {
            throw new IllegalArgumentException(
                    "tolerances must be finite and nonnegative");
        }
        if (Double.isNaN(first) || Double.isNaN(second)) {
            return false;
        }
        if (first == second) {
            return true;
        }
```

JAVA (continued 2/4)

```
        if (!Double.isFinite(first) || !Double.isFinite(second)) {
            return false;
        }
        double difference = Math.abs(first - second);
        double scale = Math.max(
                Math.abs(first), Math.abs(second));
        return difference <= Math.max(
                absoluteTolerance,
                relativeTolerance * scale);
    }

    public static int asciiDecimalDigit(char value) {
        if (value < '0' || value > '9') {
            throw new IllegalArgumentException(
                    "not an ASCII decimal digit");
        }
        return value - '0';
    }

    public static int digitInBase(char value, int base) {
        int digit = Character.digit(value, base);
        if (digit < 0) {
            throw new IllegalArgumentException(
                    "digit is not valid in base " + base);
        }
        return digit;
    }
```

JAVA (continued 3/4)

```
    public static List<Integer> sortedValues(
            List<Integer> input) {
        List<Integer> result = new ArrayList<>(input);
        result.sort(Comparator.naturalOrder());
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        Integer smallFirst = 127;
        Integer smallSecond = 127;
        Integer largeFirst = 1_000;
        Integer largeSecond = 1_000;

        System.out.println(smallFirst == smallSecond);
        System.out.println(largeFirst.equals(largeSecond));
        System.out.println(Objects.equals(null, null));

        double truncated = 7 / 2;
        double divided = 7.0 / 2;
        System.out.println(truncated); // 3.0
        System.out.println(divided);   // 3.5

        System.out.println(nearlyEqual(
                0.1 + 0.2, 0.3, 1e-12, 1e-12));

        long overflowedBeforeWidening =
                Integer.MAX_VALUE * 2;
```

JAVA (continued 4/4)

```
        long multipliedAsLong =
                Integer.MAX_VALUE * 2L;
        System.out.println(overflowedBeforeWidening);
        System.out.println(multipliedAsLong);

        System.out.println(Math.abs(Integer.MIN_VALUE));
        System.out.println(
                Math.abs((long) Integer.MIN_VALUE));
        System.out.println(-13 % 5);
        System.out.println(Math.floorMod(-13, 5));
        System.out.println(1 << 32);
        System.out.println(1L << 40);

        System.out.println(Integer.parseInt("+00042"));
        System.out.println(asciiDecimalDigit('7'));
        System.out.println(digitInBase('F', 16));

        BigDecimal exact = new BigDecimal("0.1")
                .add(new BigDecimal("0.2"));
        System.out.println(exact);

        System.out.println(sortedValues(List.of(
                Integer.MAX_VALUE,
                0,
                Integer.MIN_VALUE)));
    }
}
```

Every arithmetic example uses a type chosen to expose or prevent the intended behavior. The class does not use boxed identity as a correctness decision.

TRY IT | 15.17 Interview questions

1. Why can two equal Integer values fail an == comparison?
2. What boxing identity is guaranteed, and why should code still use equals?
3. Why does Integer.valueOf(1).equals(Long.valueOf(1)) return false?
4. Where can null unboxing occur implicitly?
5. Why is double ratio = 1 / 2 equal to 0.0?
6. Why is 0.1 + 0.2 usually not equal to 0.3 by ==?
7. What should determine a floating-point tolerance?
8. Why does casting after integer division not restore the fraction?
9. Why can an int multiplication overflow before assignment to long?
10. Why can Math.abs return a negative int?
11. How do % and Math.floorMod differ for negative values?
12. Why is 1 << 40 not a 64-bit shift?
13. What shift does 1 << 32 perform?
14. Which sign forms does Integer.parseInt accept?
15. How should an API distinguish invalid input from a valid zero?
16. When is character - '0' appropriate?
17. Why is subtraction unsafe in a comparator?
18. Why does byteValue += 1 compile while byteValue = byteValue + 1 does not?

15.18 Practice set

Answers are deliberately later in the chapter.

Quick check

1. Predict whether Integer a = 127; Integer b = 127; a == b is true.
2. Predict double value = (double) (5 / 2).
3. Predict long value = Integer.MAX_VALUE + 1.
4. Predict Math.floorMod(-1, 8).
5. Predict 1L << 65.

Coding practice

1. **Foundation:** Parse one validated ASCII decimal digit.
2. **Foundation:** Return the nonnegative magnitude of any int as long.
3. **Interview Core:** Implement safeRatio(int numerator, int denominator) with an explicit zero policy.
4. **Interview Core:** Compare two nullable Integer values with nulls last.
5. **Interview Core:** Parse an optional signed decimal int without trimming whitespace and without using Integer.parseInt.
6. **SDE-2 Follow-up:** Design a Money value object that prevents binary floating-point arithmetic.

Debugging tasks

1. Repair long area = width * height when width and height are int.
2. Repair a circular index computed as (index + delta) % length.
3. Repair a comparator that returns first.priority - second.priority.
4. Repair a nullable Integer counter increment.
5. Repair a double loop that waits for value == target after repeated additions.

Interview extension

Explain how you would review an unfamiliar numeric expression in production code. Include operand types, promotions, exceptional cases, overflow policy, rounding policy, and tests.

15.19 Delayed answer notes

Quick-check answers

1. True under the boxing identity guarantee for the constant 127.
2. 2.0. Integer division occurs before the cast.
3. -2,147,483,648 widened to long. Both addition operands are int.
5. The long distance is 65 & 63, so it is 1L << 1, which is 2.

Coding guidance

- Validate '0' <= character && character <= '9' before subtraction.
- Use Math.abs((long) value) for any int magnitude.
- Check a zero denominator before converting an operand to double.
- Comparator.nullsLast(Integer::compare) expresses nullable ordering.
- A manual parser should validate one optional sign, require at least one digit, accumulate with a range guard, and handle Integer.MIN_VALUE.
- A Money type commonly stores a scaled long under one currency and rounding contract or wraps BigDecimal with fixed scale and explicit rounding.

Debugging resolutions

Promote before multiplication: long area = (long) width * height.

For a positive length, normalize with Math.floorMod(index + delta, length). If index + delta can overflow, widen before adding or normalize each operand.

Use Integer.compare(first.priority, second.priority), then add stable tie-breakers if the domain needs them.

Validate the nullable counter before unboxing, or choose a primitive/default policy explicitly.

Do not wait for repeated floating-point addition to hit an exact target. Use a bounded iteration count, compare under a justified tolerance, or model the step count with integer units.

Review checklist answer

Read the declared operand types first. Mark every boxing, unboxing, and promotion. Determine the operation type before looking at the assignment target. Check division by zero, MIN_VALUE asymmetry, shift distance, parsing range, and null. Define overflow and rounding policies. Test zero, signs, bounds, invalid text, and values immediately around each threshold.

RECAP | 15.20 Chapter summary

- Boxed == is identity, not general numeric equality.
- Unboxing null throws at the unboxing point.
- Integral division truncates toward zero.
- A cast must occur before division to change its arithmetic type.
- Binary floating-point needs a domain-specific comparison policy.
- Expressions overflow according to operand types before assignment.
- MIN_VALUE has no positive counterpart in the same signed type.
- Java % can be negative; Math.floorMod supports nonnegative modular states.
- Shift width comes from the left operand, and distance is masked.
- Parsing must define signs, zeros, whitespace, radix, digits, and range.
- Character.digit and ASCII subtraction serve different contracts.
- Numeric comparators should use compare APIs, not subtraction.

TRY IT | 15.21 Revision checklist

[ ] I never use boxed identity as value equality.
[ ] I can find hidden unboxing.
[ ] I predict integer division before looking at the destination type.
[ ] I place casts and long operands before the operation they must affect.
[ ] I can explain floating-point tolerance as a policy.
[ ] I handle Integer.MIN_VALUE and Long.MIN_VALUE explicitly.
[ ] I distinguish remainder from floor modulus.
[ ] I know shift promotion and distance masking.
[ ] I state parsing syntax and range contracts.
[ ] I use Integer.compare, Long.compare, and Double.compare appropriately.

SDE-2 CORE CHAPTER

Chapter 4 - Chapter 15A: Expanded Practice Bank and SDE-2 Follow-Ups

This bank completes the requested assessment depth without mixing answers into the question flow. It extends Chapter 16 to forty conceptual questions, at least thirty Java-specific questions, twenty-five code-output questions, twenty-five debugging tasks, twenty algorithmic follow-ups, and ten interviewer discussion chains across Part B.

Use labels as priorities, not as judgments about difficulty. Attempt Foundation items first, then Interview Core, then SDE-2 Follow-up. Optional recreational number properties remain low priority.

15A.1 Ten additional conceptual questions

These extend Chapter 16's C01-C30.

C31

What is the difference between a number, a digit, a sign, and a textual representation?

C32

Why can leading zeros matter in a `String` even though they do not change an integer value?

C33

Why does half-reversal make a numeric palindrome check safer than full reversal?

C34

Why does a repeated-prime-query workload justify a sieve instead of repeated square-root checks?

C35

Why can factor enumeration stop at the square root, and how is a perfect square handled?

C36

Why is zero an identity for GCD reduction but an absorbing value for LCM reduction?

C37

Why do trailing zeros in a factorial count factors of five?

C38

How can logarithms count factorial digits without constructing the factorial?

C39

What condition determines whether a modular inverse exists?

C40

Why is a binary length calculation different from counting set bits?

15A.2 Thirty Java-specific retrieval questions

Answer aloud in one or two sentences before writing code.

1. Why is `long result = 100000 * 100000;` already wrong?
2. Which literal suffix makes an integer operand `long`?
3. Why does `(long) (a * b)` cast too late?
4. What type evaluates `byte + byte`?
5. When should `Math.toIntExact` replace a cast?
6. What does `Math.multiplyExact` do on overflow?
7. Why can `Math.abs(Integer.MIN_VALUE)` remain negative?
8. What is the matching `Long.MIN_VALUE` trap?
9. What sign can Java's `%` result have?
10. What range does `Math.floorMod(value, positiveModulus)` return?
11. Why can `(a, b) -> a - b` violate comparator ordering?
12. Why is `left + (right - left) / 2` contract-dependent?
13. Which method parses a `long` in a supplied radix?
14. What happens when a digit is invalid for `Integer.parseInt(text, radix)`?
15. What does `Character.digit('F', 16)` return?
16. What does `Character.forDigit(15, 16)` return?
17. Why is `char` numeric but not a general integer-storage choice?
18. Why is `0.1 + 0.2 == 0.3` unreliable?
19. Why is `BigDecimal` usually unnecessary in integer DSA problems?
20. When is `BigInteger` the correct representation rather than a workaround?
21. What is the difference between `==` and `.equals` for boxed integers?
22. What can null unboxing throw?
23. Why is `Double.NaN == Double.NaN` false?
24. How are `int` shift distances masked?
25. How are `long` shift distances masked?
26. What is the difference between `>>` and `>>>`?
27. Why does `1 << 31` not produce a positive mathematical 2^31?
28. What does `Long.numberOfLeadingZeros` help compute?

29 Why should numeric-string validation and arithmetic use the same character contract?

30 Why can a `long` product overflow even when both operands were safely normalized modulo a very large modulus?

15A.3 Five additional code-output questions

These extend Chapter 16's O01-O20. Predict the output or exception without running the code.

O21

JAVA

```java
System.out.println(100_000 * 100_000);
System.out.println(100_000L * 100_000);
```

O22

JAVA

```java
System.out.println(Math.floorMod(-17, 5));
System.out.println(-17 % 5);
```

O23

JAVA

```java
System.out.println(Long.SIZE - Long.numberOfLeadingZeros(1024));
System.out.println(Long.bitCount(1024));
```

O24

JAVA

```java
System.out.println(Character.digit('g', 16));
System.out.println(Character.forDigit(15, 16));
```

O25

JAVA

```java
System.out.println(Math.multiplyExact(Long.MAX_VALUE, 2));
```

15A.4 Five additional debugging tasks

These extend Chapter 16's D01-D20. For each: identify the bug, explain it, correct it, and state affected edge cases.

D21: Sieve-bound overflow

JAVA

```java
for (int prime = 2; prime * prime <= limit; prime++) {
    // mark multiples
}
```

D22: Numeric-string subtraction sign bug

JAVA

```java
String subtract(String left, String right) {
    return subtractMagnitudes(left, right); // assumes left >= right
}
```

D23: Trailing-zero undercount

JAVA

```java
long trailingZeros(int n) {
    return n / 5;
}
```

D24: Empty array reduction

JAVA

```java
long gcdArray(long[] values) {
    long result = values[0];
    for (long value : values) result = gcd(result, value);
    return result;
}
```

D25: Modular inverse assumption

JAVA

```java
long inverse(long value, long modulus) {
    return powerModulo(value, modulus - 2, modulus);
}
```

15A.5 Ordered implementation ladder

The following inventory gives the complete requested volume of practice. Some items also appear as worked examples in earlier chapters; here they are ordered for independent retrieval.

Thirty Foundation problems

1. **Foundation:** Count decimal digits, including zero.
2. **Foundation:** Sum decimal digits while ignoring sign.
3. **Foundation:** Multiply digits and preserve embedded-zero behavior.
4. **Foundation:** Return the minimum digit.
5. **Foundation:** Return the maximum digit.
6. **Foundation:** Count even digits.
7. **Foundation:** Count odd digits.
8. **Foundation:** Count occurrences of a target digit.
9. **Foundation:** Compute repeated digit sum.
10. **Foundation:** Reverse an integer under a documented fit assumption.
11. **Foundation:** Check a numeric palindrome by full reversal.
12. **Foundation:** Identify an Armstrong number.
13. **Foundation:** Identify a Strong number.
14. **Foundation:** Identify a Perfect number.
15. **Foundation:** Compute factorial through 20.
16. **Foundation:** Convert nonnegative decimal to binary.
17. **Foundation:** Parse a valid binary string.
18. **Foundation:** Validate binary digits.
19. **Foundation:** Convert decimal to hexadecimal.
20. **Foundation:** Parse hexadecimal.
21. **Foundation:** Apply divisibility rules for 2, 5, and 10.
22. **Foundation:** Apply divisibility rules for 3 and 9.
23. **Foundation:** Apply the alternating-sum rule for 11.
24. **Foundation:** List factor pairs.
25. **Foundation:** Count factors.
26. **Foundation:** Sum factors.
27. **Foundation:** Check primality.
28. **Foundation:** Compute GCD iteratively.
29. **Foundation:** Compute LCM with a zero contract.

30 **Foundation:** Check a positive power of two.

Thirty Interview Core problems

1. **Interview Core:** Reverse `int` with strict pre-operation checks.
2. **Interview Core:** Check palindrome by reversing only half.
3. **Interview Core:** Parse binary to `long` with overflow detection.
4. **Interview Core:** Parse huge binary with `BigInteger`.
5. **Interview Core:** Format a signed value in base 2-36.
6. **Interview Core:** Parse a signed base-2-36 value safely.
7. **Interview Core:** Validate signs and digits for a supplied base.
8. **Interview Core:** Convert arbitrary precision between bases.
9. **Interview Core:** Normalize a signed decimal string.
10. **Interview Core:** Compare signed huge decimal strings.
11. **Interview Core:** Add signed huge decimal strings.
12. **Interview Core:** Subtract signed huge decimal strings.
13. **Interview Core:** Multiply a numeric string by one digit.
14. **Interview Core:** Stream a huge-number remainder.
15. **Interview Core:** Test huge-number divisibility by 9.
16. **Interview Core:** Test huge-number divisibility by 11.
17. **Interview Core:** Return sorted factors in O(sqrt(n)).
18. **Interview Core:** Prime-factorize by trial division.
19. **Interview Core:** Generate primes with a sieve.
20. **Interview Core:** Build prefix prime counts.
21. **Interview Core:** Reduce GCD across an array.
22. **Interview Core:** Reduce LCM across an array with exact checks.
23. **Interview Core:** Normalize negative modulo.
24. **Interview Core:** Add and subtract under modulo.
25. **Interview Core:** Compute fast exact power.
26. **Interview Core:** Compute modular power.
27. **Interview Core:** Check a perfect square without overflow.
28. **Interview Core:** Compute floor square root.
29. **Interview Core:** Count binary length.

30 **Interview Core:** Count factorial trailing zeros.

Twenty SDE-2 Follow-up problems

1 **SDE-2 Follow-up:** Multiply unrestricted `long` values under modulo without overflow.

2 **SDE-2 Follow-up:** Compute a modular inverse with extended Euclid.

3 **SDE-2 Follow-up:** Explain inverse failure for non-coprime inputs.

4 **SDE-2 Follow-up:** Return factorial digit count without constructing the factorial.

5 **SDE-2 Follow-up:** Answer repeated factor-count queries from prime exponents.

6 **SDE-2 Follow-up:** Answer prime-range queries with prefix counts.

7 **SDE-2 Follow-up:** Reduce a signed fraction to canonical form.

8 **SDE-2 Follow-up:** Align repeated schedules with array LCM.

9 **SDE-2 Follow-up:** Group array values using a shared GCD.

10 **SDE-2 Follow-up:** Detect LCM overflow and return `BigInteger` when required.

11 **SDE-2 Follow-up:** Stream several moduli across one huge input.

12 **SDE-2 Follow-up:** Multiply two numeric strings.

13 **SDE-2 Follow-up:** Divide a numeric string by a small positive integer.

14 **SDE-2 Follow-up:** Convert a huge signed value between arbitrary bases.

15 **SDE-2 Follow-up:** Find the highest power of two not exceeding a capacity.

16 **SDE-2 Follow-up:** Find the next representable power-of-two capacity.

17 **SDE-2 Follow-up:** Compute an integer kth root with overflow-safe comparisons.

18 **SDE-2 Follow-up:** Compare two rational values without cross-product overflow.

19 **SDE-2 Follow-up:** Design a numeric parsing API with explicit error categories.

20 **SDE-2 Follow-up:** Defend when manual arithmetic, exact primitives, or `BigInteger` is the best contract.

15A.6 Twenty algorithmic follow-ups

Use these as interviewer changes after a working core solution.

1. The digit input changes from `int` to `Long.MIN_VALUE`.
2. Leading zeros must be preserved in output.
3. Reversal may not use a wider type.
4. Palindrome checking may inspect only half the digits.
5. Base conversion expands from base 2 to bases 2-36.
6. The source value no longer fits in `long`.
7. Numeric-string inputs may have signs and redundant zeros.
8. Numeric-string multiplication expands from one digit to another long string.
9. One prime query becomes one million bounded prime queries.
10. A factor list becomes only a factor count.
11. Factor counts are requested for many values with known prime factorizations.
12. Pairwise GCD becomes GCD of an array.
13. Pairwise LCM becomes schedule alignment over an array.
14. Modulo operands can approach `Long.MAX_VALUE`.
15. The exponent is large but nonnegative.
16. Modular division is requested.
17. Square root becomes integer kth root.
18. Factorial value is impossible to construct, but its zeros are requested.
19. Factorial digits are requested instead of its value.
20. A safe comparator must order boundary values and tolerate boxing policies.

15A.7 Five additional interviewer discussion chains

These extend Chapter 16's F01-F05.

F06: Sieve service chain

Start with one prime check. Change to repeated queries, then prime counts in ranges, then a memory-constrained upper bound. Compare trial division, sieve, prefix counts, and the boundary with segmented processing.

F07: Factorial metrics chain

Start with exact factorial. Increase `n` beyond `long`, ask for trailing zeros, then ask only for digit count. Explain why each requested output changes the representation and algorithm.

F08: Array reduction chain

Start with pairwise GCD. Generalize to an array, allow zeros and negatives, then request LCM and overflow reporting. State identities, early exits, and type policy.

F09: Modular division chain

Start with normalized remainder. Add modular multiplication, power, and inverse. Then use a composite modulus and explain why the inverse may not exist.

F10: Numeric-string API chain

Start with nonnegative comparison. Add signs, canonical zero, addition, subtraction, multiplication by a digit, and invalid-input diagnostics. Separate parsing policy from arithmetic invariants.

15A.8 Stop point

Do not continue until you have answered C31-C40, J01-J30, O21-O25, and D21-D25. For implementation problems, compare behavior on zero, signs, boundaries, malformed text, and overflow before consulting the checkpoints.

TRY IT | Part II - Delayed Checkpoints

TRY IT | 15A.9 Conceptual checkpoints

- C31-C32: a number is a value; digits and signs are representation symbols; leading zeros can encode width even when value is unchanged.
- C33: half reversal never builds the full reversed magnitude and stops after enough information exists.
- C34: a sieve pays O(n log log n) once so repeated bounded lookups become O(1).
- C35: factors pair around the square root; count an equal square-root pair once.
- C36: `gcd(0, x) = abs(x)`, while `lcm(0, x) = 0`.
- C37: tens require a two and a five; factorials contain more twos, so fives limit the count.
- C38: `log10(n!)` converts a product to a sum; floor plus one gives decimal digits.
- C39: an inverse exists exactly when `gcd(value, modulus) = 1`.
- C40: binary length counts positions through the highest one-bit; population count counts how many one-bits appear.

TRY IT | 15A.10 Java and code-output checkpoints

The Java retrieval answers should mention expression type before destination type, exact arithmetic exceptions, remainder normalization, radix validation, boxed/reference semantics, floating approximation, and fixed-width shift rules.

- O21 prints `1410065408` and then `10000000000`.
- O22 prints `3` and then `−2`.
- O23 prints `11` and then `1`.
- O24 prints `−1` and then `f`.
- O25 throws `ArithmeticException` before printing a value.

TRY IT | 15A.11 Debugging checkpoints

- D21: use `prime <= limit / prime`; the proven guard makes `prime * prime` safe for the inner start.
- D22: normalize signs and magnitudes, select addition or subtraction, and canonicalize zero.
- D23: add `n/5 + n/25 + n/125 + ...` until the divisor exceeds `n`.
- D24: reject an empty array explicitly and start a GCD fold at zero.
- D25: exponent `modulus − 2` is justified only under a suitable prime-modulus theorem; general inversion requires extended Euclid and GCD one.

15A.12 Readiness check

- **Needs foundation work:** fewer than 24 of the 40 conceptual questions can be explained without notes.
- **Foundation ready:** at least 30 conceptual answers and 20 Foundation implementations are reliable.
- **Interview ready:** all Foundation items plus at least 24 Interview Core implementations pass boundary tests.
- **Strong SDE-2 readiness:** the core set is reliable and at least 12 follow-ups can be adapted with explicit contracts, complexity, and overflow policy.

SDE-2 CORE CHAPTER

Chapter 5 - Interview Questions and Rapid Revision

This chapter is a retrieval and reasoning test, not a second pass through the explanations. Attempt the questions without running code first. Write down assumptions, predict behavior, and state complexity aloud. Then use a compiler and small tests to challenge your reasoning. The answer material begins only after every question, debugging task, coding exercise, and follow-up chain.

16.1 How to use this chapter

Use three passes:

1. **Closed-book pass:** Answer from memory. Mark uncertain items instead of guessing silently.
2. **Evidence pass:** Run snippets, write boundary tests, and compare the observed result with the language or algorithm contract.
3. **Interview pass:** Re-answer aloud in two minutes or less, including constraints, invariant, complexity, and one edge case.

Suggested scoring:

- 2 points: correct answer with a clear reason;
- 1 point: correct answer without a stable reason, or a nearly correct solution with one repair;
- 0 points: incorrect, unsafe, or unable to explain.

Do not use the answer key until all six assessment parts are complete.

TRY IT | 16.2 Part I - 30 conceptual interview questions

C01

How many decimal digits does zero have in a coding-problem contract, and why does a naive while (value > 0) loop often get this wrong?

C02

Why must an int be promoted before Math.abs when every possible int input is allowed?

C03

Why do signed two's-complement types have one more negative value than positive value?

C04

Explain why assigning an arithmetic result to long does not guarantee that the arithmetic occurred in long.

C05

What is the difference between the character '7', the int 7, and the string "7"?

C06

State the loop invariant for left-to-right base parsing with result = result * base + digit.

C07

What must be validated before accepting a digit in base b?

C08

When should leading zeros be ignored, and when might they be semantically important?

C09

Why can an arbitrarily large decimal string be tested for divisibility by nine without constructing the full integer?

C10

Explain the alternating-sum divisibility rule for eleven and why the starting sign does not affect the yes-or-no result.

C11

State Euclid's GCD invariant and its termination condition.

C12

Why is (a / gcd(a, b)) * b safer than a * b / gcd(a, b), and why is it still not automatically safe?

C13

Why can a prime test stop at sqrt(n), and how should the loop boundary be written to avoid overflow?

C14

How do factor pairs prevent duplicate work, and what special case occurs for a perfect square?

C15

Distinguish floor(log2(n)), ceil(log2(n)), and the bit length of a positive integer.

C16

Derive binary-search complexity from the size of the remaining search space.

C17

Why is mid <= n / mid preferable to mid * mid <= n in an integer square-root search?

C18

When is Math.sqrt useful for an exact integer problem, and what verification is required?

C19

Why must (value & (value - 1)) == 0 be combined with value > 0 for a power-of-two test?

C20

Why do 1 << 40 and 1L << 40 mean different operations?

C21

Contrast >> and >>> for a negative int.

C22

What shift distance does Java use for int and long, and what surprising behavior follows?

C23

Why can Java's % result be negative?

C24

When should Math.floorMod be preferred over %?

C25

Why is == unsafe as a general value comparison for Integer references even though some small boxed values share identity?

C26

Name four contexts that can unbox a nullable wrapper and throw NullPointerException.

C27

Why does casting the result of integer division differ from casting one operand before division?

C28

Why is a single universal epsilon not a sound policy for every double comparison?

C29

How do you compare two nonnegative decimal strings numerically without parsing them?

C30

When should a large-number problem use BigInteger, and when is a streaming remainder or carry state better?

16.3 Part II - 20 code-output questions

Assume each fragment appears inside a valid main method with any required java.lang types available. Predict the exact output or exception before running it.

O01

JAVA

```java
int value = Integer.MAX_VALUE;
System.out.println(value + 1);
```

O02

JAVA

```java
long value = Integer.MAX_VALUE + 1;
System.out.println(value);
```

O03

JAVA

```java
long value = (long) Integer.MAX_VALUE + 1;
System.out.println(value);
```

O04

JAVA

```java
System.out.println(7 / 2 * 2.0);
```

O05

JAVA

```java
System.out.println((double) (7 / 2));
```

O06

JAVA

```java
System.out.println(Math.abs(Integer.MIN_VALUE));
```

O07

JAVA

```java
System.out.println(-13 % 5);
System.out.println(Math.floorMod(-13, 5));
```

O08

JAVA

```java
System.out.println(1 << 32);
System.out.println(1L << 32);
```

O09

JAVA

```java
System.out.println(-1 >>> 1);
```

O10

JAVA

```java
System.out.println(-3 >> 1);
System.out.println(-3 / 2);
```

O11

JAVA

```java
Integer first = 127;
Integer second = 127;
System.out.println(first == second);
```

O12

JAVA

```java
Integer first = 1_000;
Integer second = 1_000;
System.out.println(first.equals(second));
```

O13

JAVA

```java
Integer value = null;
System.out.println(value + 1);
```

O14

JAVA

```java
System.out.println(Integer.parseInt("+007"));
```

O15

JAVA

```java
System.out.println(Character.digit('F', 16));
```

O16

JAVA

```java
byte value = 127;
value += 1;
System.out.println(value);
```

O17

JAVA

```java
double value = 0.0 / 0.0;
System.out.println(value == value);
```

O18

```java
System.out.println(Double.compare(-0.0, 0.0));
```

O19

```java
int value = 0xFFFF_FFFF;
System.out.println(value);
```

O20

```java
System.out.println("9".compareTo("10"));
```

16.4 Part III - 20 debugging questions

For each method or fragment, identify the violated contract, give a failing input, and describe a repair. Do not restrict your answer to syntax.

D01: Zero-digit bug

```java
static int countDigits(int value) {
    int count = 0;
    while (value > 0) {
        count++;
        value /= 10;
    }
    return count;
}
```

D02: Absolute-value bug

```java
static int sumDigits(int value) {
    int remaining = Math.abs(value);
    int sum = 0;
    while (remaining > 0) {
        sum += remaining % 10;
        remaining /= 10;
    }
    return sum;
}
```

D03: Reverse-overflow bug

JAVA

```java
static int reverse(int value) {
    int result = 0;
    while (value != 0) {
        result = result * 10 + value % 10;
        value /= 10;
    }
    return result;
}
```

D04: Binary-parser bug

JAVA

```java
static int binaryToInt(String text) {
    int result = 0;
    for (int i = 0; i < text.length(); i++) {
        result = result * 2 + text.charAt(i) - '0';
    }
    return result;
}
```

D05: Generic-base bug

JAVA

```java
static long parseBase(String text, int base) {
    long result = 0;
    for (int i = 0; i < text.length(); i++) {
        int digit = Character.digit(
                text.charAt(text.length() - 1 - i), base);
        result += digit * Math.pow(base, i);
    }
    return result;
}
```

D06: Signed-GCD bug

JAVA

```java
static long gcd(long first, long second) {
    long a = Math.abs(first);
    long b = Math.abs(second);
    while (b != 0) {
        long remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

D07: LCM-overflow bug

JAVA

```java
static long lcm(long first, long second) {
    return first * second / gcd(first, second);
}
```

D08: Prime-boundary bug

JAVA

```java
static boolean isPrime(int value) {
    for (int divisor = 2;
            divisor * divisor < value;
            divisor++) {
        if (value % divisor == 0) {
            return false;
        }
    }
    return true;
}
```

D09: Duplicate-factor bug

JAVA

```java
static List<Integer> factors(int value) {
    List<Integer> result = new ArrayList<>();
    for (int divisor = 1;
            divisor * divisor <= value;
            divisor++) {
        if (value % divisor == 0) {
            result.add(divisor);
            result.add(value / divisor);
        }
    }
    return result;
}
```

D10: Square-overflow bug

JAVA

```java
static boolean isPerfectSquare(int value) {
    int low = 0;
    int high = value;
    while (low <= high) {
        int mid = (low + high) / 2;
        int square = mid * mid;
        if (square == value) {
            return true;
        }
        if (square < value) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return false;
}
```

D11: Power-of-two bug

JAVA

```java
static boolean isPowerOfTwo(int value) {
    return (value & (value - 1)) == 0;
}
```

D12: Huge-number bug

JAVA

```java
static long modulo(String digits, int modulus) {
    return Long.parseLong(digits) % modulus;
}
```

D13: Final-carry bug

JAVA

```java
static String add(String first, String second) {
    int i = first.length() - 1;
    int j = second.length() - 1;
    int carry = 0;
    StringBuilder reversed = new StringBuilder();
    while (i >= 0 || j >= 0) {
        int a = i >= 0 ? first.charAt(i--) - '0' : 0;
        int b = j >= 0 ? second.charAt(j--) - '0' : 0;
        int sum = a + b + carry;
        reversed.append(sum % 10);
        carry = sum / 10;
    }
    return reversed.reverse().toString();
}
```

D14: Numeric-string comparison bug

JAVA

```java
static int compareNumbers(String first, String second) {
    return first.compareTo(second);
}
```

D15: Midpoint-overflow bug

JAVA

```java
static int midpoint(int left, int right) {
    return (left + right) / 2;
}
```

D16: Modulo-normalization bug

JAVA

```java
static int normalize(int value, int modulus) {
    return (value % modulus + modulus) % modulus;
}
```

D17: Comparator-overflow bug

JAVA

```java
values.sort((first, second) -> first - second);
```

D18: Ratio-truncation bug

JAVA

```java
static double completionRatio(int completed, int total) {
    return completed / total;
}
```

D19: Boxed-equality bug

JAVA

```java
static boolean sameCount(Integer first, Integer second) {
    return first == second;
}
```

D20: Null-unboxing bug

JAVA

```java
static void increment(Map<String, Integer> counts, String key) {
    counts.put(key, counts.get(key) + 1);
}
```

TRY IT | 16.5 Part IV - 20 short coding exercises

Write compilable Java methods. For every answer, state the numeric type, time complexity, space complexity, and at least three boundary tests.

S01 - Foundation: Count decimal digits

Implement int countDigits(int value). Zero has one digit, and the sign is not a digit. Every int input is valid.

S02 - Foundation: Digit statistics

Return the decimal digit count, sum, and product for any int in one traversal. Define the product for zero correctly.

S03 - Interview Core: Safe reverse

Implement OptionalInt reverse(int value). Return empty when the reversed mathematical value does not fit in int.

S04 - Interview Core: Numeric palindrome

Test whether a nonnegative int is a decimal palindrome without string conversion and without reversing the entire value.

S05 - Foundation: Binary validation

Implement boolean isValidBinary(String text). Require a non-null, nonempty sequence containing only ASCII '0' and '1'.

S06 - Interview Core: Binary to long

Parse a validated nonnegative binary string into long. Throw ArithmeticException when the value exceeds Long.MAX_VALUE.

S07 - Interview Core: Decimal to base

Convert a nonnegative long to a string in base 2 through 36 using uppercase letters.

S08 - Foundation: GCD

Compute the nonnegative GCD of two nonnegative long values with Euclid's algorithm.

S09 - Interview Core: Safe LCM

Return OptionalLong for the LCM of two nonnegative long values. Zero with any value has LCM zero.

S10 - Interview Core: Prime check

Test a long for primality without allowing a square-boundary multiplication to overflow.

S11 - Interview Core: Factor count

Count the positive factors of a positive long without listing all candidates through n.

S12 - Interview Core: Integer square root

Return floor(sqrt(value)) for any nonnegative long without using floating point.

S13 - Foundation: Perfect-square test

Use the integer-square-root result to test whether a long is a perfect square.

S14 - Foundation: Power of two

Test whether a signed long is a positive mathematical power of two.

S15 - Interview Core: Exact fast power

Compute base ^ exponent for a long base and nonnegative int exponent. Return empty on long overflow.

S16 - Interview Core: Huge decimal modulo

Compute a nonnegative decimal string modulo a positive int without parsing the full value.

S17 - Interview Core: Huge divisibility by eleven

Test an arbitrarily long validated decimal string with the alternating-sum rule.

S18 - Interview Core: Add numeric strings

Add two validated nonnegative decimal strings. Return canonical output without unnecessary leading zeros.

S19 - Foundation: Compare numeric strings

Compare two validated nonnegative decimal strings numerically while allowing leading zeros.

S20 - SDE-2 Follow-up: Circular index

Implement int circularIndex(int index, long delta, int length). Require 0 <= index < length and length > 0. The signed delta may be any long.

16.6 Part V - 10 medium coding problems

These problems require multiple invariants or a deliberate API policy.

M01 - Signed generic parser

Parse a string in base 2 through 36 into long. Accept one optional leading sign, reject a sign-only string, validate every digit, and support Long.MIN_VALUE exactly. Do not use Long.parseLong or BigInteger.

M02 - Signed numeric-string addition

Add two canonical or noncanonical signed decimal strings. Accept leading zeros, normalize "-0" to "0", validate syntax, and avoid BigInteger.

M03 - Numeric-string multiplication

Multiply two nonnegative decimal strings using grade-school multiplication. Validate input, canonicalize output, and analyze the O(a * b) digit work.

M04 - Integer kth root

Given nonnegative long value and int k >= 1, return floor(value ^ (1/k)). Use binary search and an overflow-safe predicate that determines whether candidate ^ k <= value.

M05 - Next power-of-two capacity

Return the smallest positive power of two greater than or equal to a positive long. Return OptionalLong.empty if no positive long answer exists.

M06 - Batch prime factorization

For many queries with values from 2 through a known limit, precompute the smallest prime factor and return each query's prime factorization efficiently. Analyze preprocessing and per-query work.

M07 - Overflow-safe modular multiplication

Compute (a * b) % modulus for 0 <= a, b < modulus and positive long modulus, even when a * b overflows long. Do not use BigInteger.

M08 - Normalized rational value

Design an immutable Rational type backed by long numerator and denominator. Normalize signs and GCD, reject denominator zero, define zero canonically, and state an overflow policy for addition and comparison.

M09 - Streaming multi-modulus scan

Read decimal digits in chunks and compute remainders for a caller-provided list of positive int moduli. Report the absolute position of an invalid character without retaining the entire input.

M10 - Binary search on a numeric answer

Workers complete tasks at known positive rates. Find the minimum whole time needed to finish at least target tasks. The feasibility sum must not overflow, and the search bound must be justified.

16.7 Part VI - Five interviewer follow-up chains

Answer each first question before revealing the next one. The chains model how an interviewer increases constraints.

F01 - Reverse integer chain

1. Reverse a positive int.
2. Add zero and negative inputs.
3. Detect int overflow.
4. Avoid a wider primitive accumulator.
5. Return a domain result that distinguishes overflow from a legitimate zero.

F02 - Base conversion chain

1. Convert a binary string to decimal.
2. Reject invalid characters.
3. Detect long overflow.
4. Generalize to bases 2 through 36.
5. Add an optional sign and support Long.MIN_VALUE.

F03 - GCD and scheduling chain

1. Implement GCD for positive ints.
2. Add zero inputs.
3. Derive LCM.
4. Detect LCM overflow.
5. Find when several recurring schedules next align and stop if the answer exceeds a deadline.

F04 - Huge-number chain

1. Compute a decimal string modulo an int.
2. Process the digits as streamed chunks.
3. Validate and report the exact invalid position.
4. Compute several moduli in one pass.
5. Parallelize chunk processing while preserving order-sensitive remainder composition.

F05 - Root and monotonic-search chain

1. Test whether an int is a perfect square.
2. Remove multiplication overflow.
3. Return floor square root for long.
4. Generalize to kth root.
5. Explain the monotonic predicate and prove the binary-search boundary updates.

16.8 Stop point

Do not continue until you have attempted:

- all 30 conceptual questions;
- all 20 code-output predictions;
- all 20 debugging diagnoses;
- at least 15 short exercises;
- at least 5 medium problems;
- all 5 follow-up chains aloud.

The remaining sections contain answers and solution guidance.

16.9 Conceptual model answers

C01

Zero has one decimal digit: 0. A loop that runs only while value > 0 executes zero times for zero, so initialize the count to one or use a do-while traversal.

C02

Math.abs(Integer.MIN_VALUE) cannot produce a positive int because 2^31 is outside the positive int range. Convert to long first and then take the absolute value.

C03

With w bits, two's complement represents values from -2^(w-1) through 2^(w-1) - 1. Zero occupies one nonnegative pattern, leaving one extra negative magnitude.

C04

Binary numeric promotion uses operand types to select the operation type. If both operands are int, overflow occurs in int; assignment conversion to long happens only after that result exists.

C05

'7' is a char code unit, 7 is an int numeric value, and "7" is a String containing one character. Converting among them requires validation and an explicit numeric or textual rule.

C06

Before each iteration, result equals the numeric value represented by the processed prefix. Multiplying by base shifts that prefix one digit left, and adding the new digit extends the invariant.

C07

Validate that the base is supported, the text is nonempty under the sign policy, and Character.digit(character, base) returns a nonnegative value. Also validate overflow before multiplying and adding.

C08

Ignore leading zeros when comparing or canonicalizing numeric values. Preserve them when the data is an identifier, fixed-width field, account code, or textual representation whose formatting is part of the contract.

C09

Congruence is preserved by decimal extension. If two prefixes have the same remainder modulo nine, multiplying both by ten and adding the same digit preserves equivalent remainders, so only the current remainder is needed.

C10

A decimal value is divisible by eleven when the alternating signed digit sum is divisible by eleven. Reversing every sign negates the sum, and a value is divisible by eleven exactly when its negation is too.

C11

gcd(a, b) equals gcd(b, a % b). Repeated replacement reduces the second nonnegative operand until it is zero; the remaining first operand is the GCD.

C12

Dividing by the GCD first reduces an operand before multiplication, avoiding some unnecessary overflow. The reduced product can still exceed long, so compare against Long.MAX_VALUE / otherOperand or use Math.multiplyExact.

C13

If n = a * b and a is greater than sqrt(n), then b is less than sqrt(n), so one factor would already have been found. Use divisor <= n / divisor rather than divisor * divisor <= n.

C14

Each divisor d produces paired divisor n / d. At an exact square root, the two members are equal and must be counted or emitted once.

C15

floor(log2(n)) is the highest set-bit index for positive n. ceil(log2(n)) is the number of doublings from one needed to reach at least n. Bit length is floor(log2(n)) + 1.

C16

After k valid binary-search decisions, at most about n / 2 ^ k candidates remain. Requiring that quantity to reach one gives k in O(log2(n)).

C17

mid * mid can overflow even when mid and n are valid long values. For positive mid, mid <= n / mid expresses the same ordering without multiplication.

C18

Math.sqrt is a useful constant-time estimate or initial candidate. For exact long behavior, adjust and verify the candidate with division and remainder, or use an integer binary search for a proof independent of floating-point rounding.

C19

Zero passes the bit expression because 0 & -1 is zero. Negative values such as Long.MIN_VALUE can also contain one set bit, so require a positive mathematical value.

C20

The left operand determines the promoted width. The first expression shifts a 32-bit int with distance masked to five bits; the second shifts a 64-bit long with distance masked to six bits.

C21

Signed right shift >> fills new high bits with the sign bit. Unsigned right shift >>> fills them with zero, so a negative int becomes a nonnegative bit-pattern interpretation after enough shifting.

C22

int uses the low five bits of the distance, equivalent to distance & 31. long uses the low six bits, equivalent to distance & 63. Therefore shifting int by 32 or long by 64 acts like shifting by zero.

C23

Java defines % as remainder after division that truncates toward zero. The remainder has the dividend's sign or is zero, so a negative dividend can produce a negative remainder.

C24

Use Math.floorMod when the domain needs a result in [0, modulus) for a positive modulus, such as circular indices, clock arithmetic, and normalized hash buckets.

C25

When both operands are references, == compares identity. Small boxed constants have selected identity guarantees, but value correctness must not depend on allocation or cache behavior; use equals with a null policy.

C26

Arithmetic, comparison with a primitive, assignment to a primitive, and method invocation requiring a primitive can unbox. switch and selected conditional-expression contexts can also unbox.

C27

Casting the result occurs after integer division has already discarded the fraction. Casting one operand first makes binary numeric promotion select floating-point division.

C28

An acceptable error depends on scale, units, algorithm, and accumulated operations. A fixed absolute epsilon may be too large near zero or too small for large magnitudes; often a documented combination of absolute and relative tolerance is needed.

C29

Validate and remove unnecessary leading zeros. Compare canonical lengths first; if lengths match, compare digit characters lexicographically.

C30

Use BigInteger when the exact entire value must be retained and arbitrary-precision operations are part of the solution. Use a streaming state when the question needs only a remainder, checksum, comparison summary, or carry, because it is simpler and uses bounded extra space.

16.10 Code-output answer key

O01

Output: -2147483648. The int addition wraps from Integer.MAX_VALUE to Integer.MIN_VALUE.

O02

Output: -2147483648. Both operands are int, so the overflow occurs before widening to long.

O03

Output: 2147483648. Casting the first operand makes the addition occur in long.

O04

Output: 6.0. The subexpression 7 / 2 evaluates to int 3, and 3 * 2.0 is double 6.0.

O05

Output: 3.0. Integer division completes before the cast.

O06

Output: -2147483648. The positive magnitude does not fit in int.

O07

Output:

```text
-3
2
```

Remainder follows the negative dividend; floorMod normalizes for positive modulus.

O08

Output:

```text
1
4294967296
```

The int shift distance 32 is masked to zero. The long expression represents 2 ^ 32.

O09

Output: 2147483647. Unsigned right shift fills the high bit with zero.

O10

Output:

```text
-2
-1
```

Signed right shift extends the sign and behaves like floor division by two here. Integer division truncates toward zero.

O11

Output: true. Boxing the constant int 127 is within the required small-value identity range.

O12

Output: true. Integer.equals compares same-wrapper numeric value; identity is irrelevant.

O13

Result: NullPointerException before println can print a value. Addition requires unboxing null.

O14

Output: 7. parseInt accepts one leading plus sign and decimal leading zeros.

O15

Output: 15.

O16

Output: -128. Compound assignment includes an implicit narrowing cast back to byte after int addition.

O17

Output: false. NaN is unequal to every value, including itself.

O18

Output: -1. Double.compare defines negative zero as ordered before positive zero.

O19

Output: -1. The hexadecimal literal denotes the 32-bit int pattern with every bit set.

O20

Output: 8, the character-code difference between '9' and '1'. Raw lexicographic order does not equal numeric order for different digit lengths.

16.11 Debugging answer guide

D01

Failing inputs include 0 and every negative value. Zero returns zero digits, while negatives never enter the loop. Promote to long, take the magnitude, and use a do-while or initialize the count for zero.

D02

Integer.MIN_VALUE remains negative after Math.abs, and zero never enters the loop. Use long remaining = Math.abs((long) value) and a do-while traversal.

D03

An input such as 1,534,236,469 reverses beyond the int range and wraps during reconstruction. Accumulate an int input in long and range-check before narrowing, or guard each multiply-add before it occurs.

D04

The method accepts characters other than '0' and '1', rejects neither null nor empty input deliberately, and can overflow int. Validate the string and check result > (Integer.MAX_VALUE - bit) / 2 before updating.

D05

Character.digit can return -1, the base can be invalid, Math.pow returns double, casting loses exactness, and the sum can overflow. Scan left to right with checked Horner accumulation after validating base and every digit.

D06

Math.abs(Long.MIN_VALUE) remains negative, so the claimed nonnegative normalization fails. Either require nonnegative inputs, use BigInteger for every signed long pair, or return a result type capable of representing the unsigned magnitude 2^63.

D07

first * second can overflow before division, and gcd(0, 0) may lead to division by zero. Handle zero first, divide one input by GCD, then validate or exactly perform the remaining multiplication.

D08

The method returns true for values below two. The strict boundary misses exact square divisors, such as 3 for 9, and divisor * divisor can overflow. Reject values below two and loop while divisor <= value / divisor.

D09

A perfect square adds the square root twice. The method also lacks a positive-value contract, can overflow divisor * divisor, and does not produce sorted order. Add the paired factor only when paired != divisor and use divisor <= value / divisor.

D10

Negative values create an invalid range. `low + high` and `mid * mid` can overflow, causing wrong direction changes. Reject negative input, return `true` immediately for zero and one, then search from `low = 1` with `low + (high - low) / 2` and compare `mid <= value / mid`. Handling zero before division prevents a `0 / 0` failure.

D11

Zero passes because 0 & -1 is zero, and Integer.MIN_VALUE also has one set bit despite being negative. Require value > 0.

D12

Long.parseLong fails for the exact inputs that motivate a large-number-string method. It also lacks digit and modulus validation. Scan characters and carry remainder = (remainder * 10 + digit) % modulus.

D13

"999" + "1" ends with carry one after both indices are exhausted, but the loop stops and returns "000". Continue while i >= 0 || j >= 0 || carry != 0, and validate digits before arithmetic.

D14

"9".compareTo("10") is positive even though nine is numerically smaller. Validate and remove leading zeros, compare lengths, then compare equal-length canonical strings.

D15

Large positive indices can overflow left + right. Under the array-index contract 0 <= left <= right, return left + (right - left) / 2.

D16

The method does not require a positive modulus, division by zero is possible, and value % modulus + modulus can overflow int for a very large modulus. Validate modulus > 0 and use Math.floorMod(value, modulus).

D17

For Integer.MIN_VALUE and a positive value, subtraction can overflow and return a sign that reverses the order. Use Integer.compare(first, second) or Comparator.naturalOrder().

D18

Both operands are int, so fractional information is discarded. Validate total according to the zero policy and cast an operand before division: (double) completed / total.

D19

The method compares wrapper identity and can also produce false when equal values are distinct objects. Choose Objects.equals for nullable same-wrapper equality, or validate non-null values and compare their unboxed ints.

D20

counts.get(key) returns null for an absent key, and addition unboxes it. Use counts.merge(key, 1, Integer::sum), getOrDefault, or an explicit initialization policy. Consider Math.addExact if count overflow matters.

16.12 Short-exercise reference solutions

The following class supplies one compiling reference implementation for S01 through S20. Method names correspond to the exercise order. Alternative solutions are valid when their contracts and boundary behavior are equally explicit.

JAVA (continued 1/11)

```java
import java.util.OptionalInt;
import java.util.OptionalLong;

public final class RapidRevisionSolutions {
    private static final char[] BASE_DIGITS =
            "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();

    private RapidRevisionSolutions() {
    }

    public record DigitStats(
            int count, int sum, long product) {
    }

    public static int countDigits(int value) {
        long remaining = Math.abs((long) value);
        int count = 0;
        do {
            count++;
            remaining /= 10;
        } while (remaining > 0);
        return count;
    }

    public static DigitStats digitStats(int value) {
        long remaining = Math.abs((long) value);
        int count = 0;
        int sum = 0;
        long product = 1;
        do {
            int digit = (int) (remaining % 10);
            count++;
            sum += digit;
            product *= digit;
            remaining /= 10;
```

JAVA (continued 2/11)

```
        } while (remaining > 0);
        return new DigitStats(count, sum, product);
    }

    public static OptionalInt reverse(int value) {
        long remaining = Math.abs((long) value);
        long reversed = 0;
        do {
            reversed = reversed * 10 + remaining % 10;
            remaining /= 10;
        } while (remaining > 0);
        long signed = value < 0 ? -reversed : reversed;
        if (signed < Integer.MIN_VALUE
                || signed > Integer.MAX_VALUE) {
            return OptionalInt.empty();
        }
        return OptionalInt.of((int) signed);
    }

    public static boolean isPalindrome(int value) {
        if (value < 0 || value != 0 && value % 10 == 0) {
            return false;
        }
        int prefix = value;
        int suffix = 0;
        while (prefix > suffix) {
            suffix = suffix * 10 + prefix % 10;
            prefix /= 10;
        }
        return prefix == suffix || prefix == suffix / 10;
    }

    public static boolean isValidBinary(String text) {
        if (text == null || text.isEmpty()) {
            return false;
```

JAVA (continued 3/11)

```
        }
        for (int i = 0; i < text.length(); i++) {
            char current = text.charAt(i);
            if (current != '0' && current != '1') {
                return false;
            }
        }
        return true;
    }

    public static long binaryToLong(String text) {
        if (!isValidBinary(text)) {
            throw new IllegalArgumentException(
                    "invalid binary text");
        }
        long result = 0;
        for (int i = 0; i < text.length(); i++) {
            int bit = text.charAt(i) - '0';
            if (result > (Long.MAX_VALUE - bit) / 2) {
                throw new ArithmeticException(
                        "binary value exceeds long");
            }
            result = result * 2 + bit;
        }
        return result;
    }

    public static String toBase(long value, int base) {
        if (value < 0) {
            throw new IllegalArgumentException(
                    "value must be nonnegative");
        }
        if (base < 2 || base > 36) {
            throw new IllegalArgumentException(
                    "base must be between 2 and 36");
```

JAVA (continued 4/11)

```
        }
        if (value == 0) {
            return "0";
        }
        StringBuilder reversed = new StringBuilder();
        long remaining = value;
        while (remaining > 0) {
            int digit = (int) (remaining % base);
            reversed.append(BASE_DIGITS[digit]);
            remaining /= base;
        }
        return reversed.reverse().toString();
    }

    public static long gcd(long first, long second) {
        requireNonnegative(first, "first");
        requireNonnegative(second, "second");
        long a = first;
        long b = second;
        while (b != 0) {
            long remainder = a % b;
            a = b;
            b = remainder;
        }
        return a;
    }

    public static OptionalLong safeLcm(
            long first, long second) {
        requireNonnegative(first, "first");
        requireNonnegative(second, "second");
        if (first == 0 || second == 0) {
            return OptionalLong.of(0);
        }
        long reduced = first / gcd(first, second);
```

JAVA (continued 5/11)

```
        if (reduced > Long.MAX_VALUE / second) {
            return OptionalLong.empty();
        }
        return OptionalLong.of(reduced * second);
    }

    public static boolean isPrime(long value) {
        if (value < 2) {
            return false;
        }
        if (value == 2) {
            return true;
        }
        if (value % 2 == 0) {
            return false;
        }
        for (long divisor = 3;
                divisor <= value / divisor;
                divisor += 2) {
            if (value % divisor == 0) {
                return false;
            }
        }
        return true;
    }

    public static int factorCount(long value) {
        if (value <= 0) {
            throw new IllegalArgumentException(
                    "value must be positive");
        }
        int count = 0;
        for (long divisor = 1;
                divisor <= value / divisor;
                divisor++) {
```

JAVA (continued 6/11)

```
            if (value % divisor == 0) {
                count += divisor == value / divisor ? 1 : 2;
            }
        }
        return count;
    }

    public static long floorSquareRoot(long value) {
        requireNonnegative(value, "value");
        if (value < 2) {
            return value;
        }
        long low = 1;
        long high = Math.min(
                value / 2 + 1, 3_037_000_499L);
        long answer = 1;
        while (low <= high) {
            long mid = low + (high - low) / 2;
            if (mid <= value / mid) {
                answer = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return answer;
    }

    public static boolean isPerfectSquare(long value) {
        if (value < 0) {
            return false;
        }
        long root = floorSquareRoot(value);
        return root == 0
```

JAVA (continued 7/11)

```
                ? value == 0
                : value / root == root
                        && value % root == 0;
    }

    public static boolean isPowerOfTwo(long value) {
        return value > 0 && (value & (value - 1)) == 0;
    }

    public static OptionalLong safePower(
            long base, int exponent) {
        if (exponent < 0) {
            throw new IllegalArgumentException(
                    "exponent must be nonnegative");
        }
        long result = 1;
        long factor = base;
        int remaining = exponent;
        while (remaining > 0) {
            if ((remaining & 1) != 0) {
                OptionalLong product =
                        exactProduct(result, factor);
                if (product.isEmpty()) {
                    return OptionalLong.empty();
                }
                result = product.getAsLong();
            }
            remaining >>>= 1;
            if (remaining > 0) {
                OptionalLong square =
                        exactProduct(factor, factor);
                if (square.isEmpty()) {
                    return OptionalLong.empty();
                }
```

JAVA (continued 8/11)

```
                factor = square.getAsLong();
            }
        }
        return OptionalLong.of(result);
    }

    public static int largeModulo(
            String digits, int modulus) {
        requireDecimalDigits(digits);
        if (modulus <= 0) {
            throw new IllegalArgumentException(
                    "modulus must be positive");
        }
        long remainder = 0;
        for (int i = 0; i < digits.length(); i++) {
            int digit = digits.charAt(i) - '0';
            remainder = (remainder * 10 + digit) % modulus;
        }
        return (int) remainder;
    }

    public static boolean isDivisibleBy11(String digits) {
        requireDecimalDigits(digits);
        int alternating = 0;
        int sign = 1;
        for (int i = 0; i < digits.length(); i++) {
            int digit = digits.charAt(i) - '0';
            alternating = (alternating + sign * digit) % 11;
            sign = -sign;
        }
        return alternating == 0;
    }

    public static String add(
```

JAVA (continued 9/11)

```
            String first, String second) {
        requireDecimalDigits(first);
        requireDecimalDigits(second);
        int i = first.length() - 1;
        int j = second.length() - 1;
        int carry = 0;
        StringBuilder reversed = new StringBuilder();
        while (i >= 0 || j >= 0 || carry != 0) {
            int a = i >= 0 ? first.charAt(i--) - '0' : 0;
            int b = j >= 0 ? second.charAt(j--) - '0' : 0;
            int sum = a + b + carry;
            reversed.append((char) ('0' + sum % 10));
            carry = sum / 10;
        }
        return canonical(reversed.reverse().toString());
    }

    public static int compare(
            String first, String second) {
        String a = canonical(first);
        String b = canonical(second);
        if (a.length() != b.length()) {
            return Integer.compare(a.length(), b.length());
        }
        return a.compareTo(b);
    }

    public static int circularIndex(
            int index, long delta, int length) {
        if (length <= 0 || index < 0 || index >= length) {
            throw new IllegalArgumentException(
                    "require length > 0 and valid index");
        }
        long normalizedDelta = Math.floorMod(
```

JAVA (continued 10/11)

```
                delta, (long) length);
        return (int) ((index + normalizedDelta) % length);
    }

    private static OptionalLong exactProduct(
            long first, long second) {
        try {
            return OptionalLong.of(
                    Math.multiplyExact(first, second));
        } catch (ArithmeticException overflow) {
            return OptionalLong.empty();
        }
    }

    private static String canonical(String digits) {
        requireDecimalDigits(digits);
        int firstNonzero = 0;
        while (firstNonzero < digits.length() - 1
                && digits.charAt(firstNonzero) == '0') {
            firstNonzero++;
        }
        return digits.substring(firstNonzero);
    }

    private static void requireDecimalDigits(String digits) {
        if (digits == null || digits.isEmpty()) {
            throw new IllegalArgumentException(
                    "digits must not be empty");
        }
        for (int i = 0; i < digits.length(); i++) {
            char current = digits.charAt(i);
            if (current < '0' || current > '9') {
                throw new IllegalArgumentException(
                        "invalid digit at index " + i);
```

JAVA (continued 11/11)

```
            }
        }
    }

    private static void requireNonnegative(
            long value, String name) {
        if (value < 0) {
            throw new IllegalArgumentException(
                    name + " must be nonnegative");
        }
    }

    public static void main(String[] args) {
        System.out.println(countDigits(Integer.MIN_VALUE));
        System.out.println(digitStats(1_203));
        System.out.println(reverse(-120));
        System.out.println(isPalindrome(12_321));
        System.out.println(binaryToLong("1011"));
        System.out.println(toBase(255, 16));
        System.out.println(gcd(48, 18));
        System.out.println(safeLcm(21, 6));
        System.out.println(isPrime(29));
        System.out.println(factorCount(36));
        System.out.println(floorSquareRoot(27));
        System.out.println(isPerfectSquare(81));
        System.out.println(isPowerOfTwo(1_024));
        System.out.println(safePower(3, 5));
        System.out.println(largeModulo("1234", 7));
        System.out.println(isDivisibleBy11("121"));
        System.out.println(add("999", "1"));
        System.out.println(compare("0009", "10"));
        System.out.println(circularIndex(1, Long.MIN_VALUE, 7));
    }
}
```

Complexity and boundary table

Exercises	Time	Extra space excluding output	Critical tests
S01-S04	O(decimal digits)	O(1)	0, negative, MIN_VALUE, overflow
S05	O(d)	O(1)	null, empty, "0", invalid char
S06	O(d)	O(1)	leading zeros, MAX_VALUE, one-bit overflow
S07	O(log_base n)	O(output)	zero, base 2, base 36
S08-S09	O(log min(a,b))	O(1)	both zero, one zero, LCM overflow
S10-S11	O(sqrt(n))	O(1)	below two, square, large prime
S12-S13	O(log n)	O(1)	0, 1, MAX_VALUE
S14	O(1)	O(1)	negative, zero, highest positive power
S15	O(log exponent)	O(1)	exponent 0, negative base, overflow
S16-S17	O(d)	O(1)	huge text, zeros, invalid digit
S18-S19	O(max(a,b))	O(output)	leading zeros, carry, equal values
S20	O(1)	O(1)	negative delta, MIN_VALUE, length 1

16.13 Medium-problem solution guide

M01 solution: Signed generic parser

Accumulate the result as a negative long. The negative range can represent Long.MIN_VALUE directly, while the positive range cannot represent its magnitude. After processing an optional sign, set:

```text
limit = Long.MIN_VALUE       for a negative input
limit = -Long.MAX_VALUE      for a positive input
```

Before multiplying, require result >= limit / base. Before subtracting the next digit, require multipliedResult >= limit + digit. Return the negative result directly for a negative input and negate it for a positive input.

JAVA (continued 1/2)

```java
public final class SignedRadixParser {
    private SignedRadixParser() {
    }

    public static long parse(String text, int base) {
        if (base < 2 || base > 36) {
            throw new IllegalArgumentException("invalid base");
        }
        if (text == null || text.isEmpty()) {
            throw new IllegalArgumentException("empty input");
        }

        int index = 0;
        boolean negative = false;
        char first = text.charAt(0);
        if (first == '+' || first == '-') {
            negative = first == '-';
            index++;
        }
        if (index == text.length()) {
            throw new IllegalArgumentException("sign without digits");
        }

        long limit = negative
                ? Long.MIN_VALUE
                : -Long.MAX_VALUE;
        long multiplicationLimit = limit / base;
        long result = 0;
```

JAVA (continued 2/2)

```java
        while (index < text.length()) {
            int digit = Character.digit(
                    text.charAt(index), base);
            if (digit < 0) {
                throw new IllegalArgumentException(
                        "invalid digit at index " + index);
            }
            if (result < multiplicationLimit) {
                throw new ArithmeticException("long overflow");
            }
            result *= base;
            if (result < limit + digit) {
                throw new ArithmeticException("long overflow");
            }
            result -= digit;
            index++;
        }
        return negative ? result : -result;
    }

    public static void main(String[] args) {
        System.out.println(parse(
                "-8000000000000000", 16));
        System.out.println(parse(
                "7FFFFFFFFFFFFFFF", 16));
    }
}
```

Complexity: O(d) time and O(1) space.

Critical tests: "+0", "-0", sign only, invalid digit for base, Long.MAX_VALUE, Long.MIN_VALUE, and one step beyond each bound.

M02 solution: Signed numeric-string addition

Parse each input into sign and canonical magnitude. If signs match, add magnitudes and retain the sign. If signs differ, compare magnitudes, subtract the smaller from the larger, and use the larger magnitude's sign. Canonicalize every zero result to "0".

Magnitude addition is the carry scan from Chapter 14. Magnitude subtraction scans right to left with a borrow and requires firstMagnitude >= secondMagnitude.

Complexity: O(max(a, b)) time and output space.

Critical invariant: Before each subtraction position, borrow is zero or one, and all less-significant output digits are final.

M03 solution: Numeric-string multiplication

Validate and canonicalize both inputs. If either is zero, return "0". Allocate an int array of a.length + b.length. For each pair of digits from right to left, add their product to the lower result position, store sum % 10 there, and carry sum / 10 into the next position to the left.

JAVA

```java
static String multiply(String first, String second) {
    String a = canonicalDigits(first);
    String b = canonicalDigits(second);
    if (a.equals("0") || b.equals("0")) {
        return "0";
    }

    int[] result = new int[a.length() + b.length()];
    for (int i = a.length() - 1; i >= 0; i--) {
        int leftDigit = a.charAt(i) - '0';
        for (int j = b.length() - 1; j >= 0; j--) {
            int rightDigit = b.charAt(j) - '0';
            int low = i + j + 1;
            int high = i + j;
            int sum = leftDigit * rightDigit + result[low];
            result[low] = sum % 10;
            result[high] += sum / 10;
        }
    }

    StringBuilder output = new StringBuilder(result.length);
    int index = result[0] == 0 ? 1 : 0;
    while (index < result.length) {
        output.append((char) ('0' + result[index++]));
    }
    return output.toString();
}
```

The helper canonicalDigits must validate a nonempty ASCII decimal string and remove unnecessary leading zeros.

Complexity: O(a * b) time and O(a + b) space.

M04 solution: Integer kth root

Binary-search candidate from zero through min(value, a safe upper bound). The monotonic predicate is candidate ^ k <= value. Evaluate it with an accumulator and stop before multiplication when accumulator > value / candidate.

For candidate zero, the predicate is true for nonnegative value. For k = 1, the answer is value.

Complexity: O(k log value) with repeated multiplication in each predicate. Exponentiation by squaring can reduce each predicate to O(log k), but needs equally careful overflow capping.

M05 solution: Next power-of-two capacity

Start at one and double while capacity < value. Before doubling, check capacity > Long.MAX_VALUE / 2. The largest positive power of two representable in long is 2 ^ 62.

Complexity: O(log value) time and O(1) space.

Critical tests: 1, an existing power, one above a power, 2 ^ 62, and 2 ^ 62 + 1.

M06 solution: Batch prime factorization

Build an int array smallestPrimeFactor from zero through limit. When an unmarked i is found, set its own factor to i and mark still-unmarked multiples beginning at i * i. Use i <= limit / i to avoid square overflow.

To factor query q, repeatedly emit smallestPrimeFactor[q] and divide q by it until q becomes one.

Complexity: About O(limit log log limit) preprocessing, O(limit) memory, and O(number of prime factors with repetition) per query.

M07 solution: Overflow-safe modular multiplication

Use binary decomposition of b. Maintain result and factor in [0, modulus). Modular addition avoids x + y overflow by comparing x with modulus - y.

JAVA (continued 1/2)

```
public final class ModularMultiplication {
    private ModularMultiplication() {
    }

    public static long multiplyMod(
            long first, long second, long modulus) {
        if (modulus <= 0
                || first < 0 || first >= modulus
                || second < 0 || second >= modulus) {
            throw new IllegalArgumentException(
                    "require 0 <= operands < positive modulus");
        }
        long result = 0;
        long factor = first;
        long multiplier = second;
        while (multiplier > 0) {
            if ((multiplier & 1) != 0) {
                result = addMod(result, factor, modulus);
            }
            multiplier >>>= 1;
```

JAVA (continued 2/2)

```
            if (multiplier > 0) {
                factor = addMod(factor, factor, modulus);
            }
        }
        return result;
    }

    private static long addMod(
            long first, long second, long modulus) {
        return first >= modulus - second
                ? first - (modulus - second)
                : first + second;
    }

    public static void main(String[] args) {
        long modulus = Long.MAX_VALUE - 58;
        System.out.println(multiplyMod(
                modulus - 2, modulus - 3, modulus));
    }
}
```

Complexity: O(log second) time and O(1) space.

M08 solution: Normalized rational value

Keep final fields long numerator and long denominator, with denominator strictly positive and gcd equal to one. Canonical zero is 0/1.

A robust overflow policy is:

- perform constructor sign normalization and GCD in BigInteger;
- convert normalized fields with longValueExact, rejecting a rational whose canonical long fields do not fit;
- perform addition and comparison cross-products in BigInteger;
- normalize the addition result back through the constructor.

This preserves long-backed storage while making overflow a checked API outcome rather than silent wraparound.

Complexity: Depends on operand bit length; for fixed-width inputs, treat the number of machine words as bounded but still state that BigInteger allocates.

M09 solution: Streaming multi-modulus scan

Copy and validate the moduli once. Keep one long remainder per modulus and a long absolutePosition. For each ASCII digit, update every remainder with (remainder * 10 + digit) % modulus. Because each modulus is int, the intermediate fits in long.

At chunk boundaries, retain the remainders and position; no digit history is needed.

Complexity: O(d * m) time and O(m) space for d digits and m moduli.

M10 solution: Binary search on a numeric answer

First clarify units. A common form supplies each worker's positive duration per task. At time t, that worker finishes t / duration tasks.

The feasibility predicate sums contributions but returns true as soon as total >= target. To avoid sum overflow, add only up to the remaining target or use:

JAVA

```java
if (completed >= target - contribution) {
    return true;
}
completed += contribution;
```

Find an upper bound by doubling time until feasible, checking overflow. Then binary-search the first feasible time with low + (high - low) / 2.

Complexity: O(workers * log answer) time and O(1) extra space.

Critical invariant: All times below low are infeasible, and high is feasible.

TRY IT | 16.14 Follow-up-chain model checkpoints

F01 checkpoints: Reverse integer

1. Repeatedly take digit = value % 10 and remove it with value /= 10.
2. Decide whether to process signed remainders directly or promote the magnitude to long. Zero must produce zero.
3. A long accumulator is sufficient for int input; apply the sign and range-check before narrowing.
4. Without a wider accumulator, check the current result against Integer.MAX_VALUE / 10 and Integer.MIN_VALUE / 10 before result * 10 + digit. At equal boundaries, the final positive digit may be at most 7 and the final negative digit at least -8.
5. OptionalInt, a sealed domain result, or an exception distinguishes overflow from a valid reversed zero. State which policy the caller needs.

F02 checkpoints: Base conversion

1. Horner accumulation processes each bit with result = result * 2 + bit.
2. Validate null, empty input, and every character before using its numeric value.
3. Before an update, require result <= (Long.MAX_VALUE - digit) / base.
4. Validate base 2 through 36 and use Character.digit.
5. Accumulate negatively under a sign-specific limit so Long.MIN_VALUE is representable. Reject a sign-only input.

F03 checkpoints: GCD and scheduling

1. Use Euclid's replacement (a, b) = (b, a % b).
2. gcd(a, 0) is a, and this contract defines gcd(0, 0) as zero.
3. For nonzero inputs, lcm = (a / gcd(a, b)) * b.
4. Check the final product or use Math.multiplyExact. Handle zero before dividing.
5. Fold schedules one at a time with safe LCM. Stop immediately if the partial alignment exceeds the deadline or becomes unrepresentable.

F04 checkpoints: Huge-number processing

1. Carry only the prefix remainder.
2. Persist remainder and absolute position across chunks.
3. Validate each character before arithmetic and report position + localIndex.
4. Keep one remainder per modulus; time becomes O(d * m).
5. For ordered chunk composition, summarize each chunk by its remainder and digit length. Combine left and right as (leftRemainder * 10^rightLength + rightRemainder) % modulus using overflow-safe modular arithmetic. The reduction tree must preserve chunk order.

F05 checkpoints: Root search

1. Reject negative values and verify an integer candidate.
2. Compare candidate <= value / candidate.
3. Binary-search the last true candidate in a nonnegative long range.
4. Define predicate powerAtMost(candidate, k, value), stopping before a multiply that would exceed value.
5. The predicate is monotonic: once candidate^k exceeds value, every larger nonnegative candidate also exceeds it. Preserve the last true answer and move low or high without discarding a possible boundary.

16.15 Final revision cheat sheet

Java integer ranges

Type	Width	Minimum	Maximum
byte	8	-2^7	2^7 - 1
short	16	-2^15	2^15 - 1
int	32	-2^31	2^31 - 1
long	64	-2^63	2^63 - 1

Remember:

- promote an int before Math.abs when MIN_VALUE is possible;
- 1L, not 1, selects a long shift;
- int arithmetic can overflow before long assignment;
- exact APIs throw rather than wrap.

Powers of two

Power	Decimal
2^10	1,024
2^20	1,048,576
2^30	1,073,741,824
2^31	2,147,483,648
2^32	4,294,967,296
2^62	4,611,686,018,427,387,904
2^63	9,223,372,036,854,775,808

2 ^ 30 is the largest positive power of two that fits in int. 2 ^ 62 is the largest positive power of two that fits in long.

Core arithmetic identities

TEXT

```
last decimal digit        n % 10
remove last digit         n / 10
Horner parse              result * base + digit
GCD step                  gcd(a, b) = gcd(b, a % b)
safe LCM shape            (a / gcd(a, b)) * b
factor boundary           divisor <= n / divisor
safe square boundary      root <= n / root
safe index midpoint       left + (right - left) / 2
power of two              n > 0 && (n & (n - 1)) == 0
large decimal remainder   (remainder * 10 + digit) % modulus
```

Logarithm intuition

Repeated change	Rounds
Remove one item	O(n)
Halve remaining work	O(log n)
Process n items at each halving level	O(n log n)
Make two choices for each item	O(2^n)

For positive n:

```text
highest set-bit index = floor(log2(n))
bit length            = floor(log2(n)) + 1
doublings to reach n  = ceil(log2(n))
```

Shift rules

Expression	Behavior
value << k	Shift left; discarded high bits do not throw
value >> k	Signed right shift with sign extension
value >>> k	Unsigned right shift with zero fill
int distance	k & 31
long distance	k & 63

byte, short, and char promote to int before shifting.

Parsing and character rules

- Integer.parseInt accepts one leading + or - and no automatic trimming.
- Leading zeros remain decimal for parseInt(String).
- Validate Character.digit(character, base) >= 0.
- character - '0' is for validated ASCII decimal digits.
- Detect parse overflow before multiply-add.
- Decide whether signs, prefixes, whitespace, separators, and Unicode digits are allowed.

Equality and comparison

Need	Use
Primitive int equality	==
Same-wrapper value equality	equals with null policy
Nullable wrapper equality	Objects.equals
int ordering	Integer.compare
long ordering	Long.compare
double ordering	Double.compare
Approximate measurement equality	documented absolute/relative tolerance

Do not sort fixed-width integers by subtraction.

Modulo rules

TEXT

```
-13 % 5                = -3
Math.floorMod(-13, 5)  = 2
```

Use floorMod for a nonnegative state under a positive modulus. For huge decimal strings, carry only the remainder.

Problem-recognition map

Signal	First pattern to consider
Repeated decimal digits	quotient/remainder by 10
Base-b text	Horner accumulation
Output in base b	repeated division and reverse
Divisors or prime	factor pairs through sqrt(n)
Repeating cycles	GCD or safe LCM
Huge number, only divisibility	streaming remainder
Huge number addition	right-to-left carry
Huge number comparison	canonical length then lexicographic
Exact primitive arithmetic	pre-check or Math.*Exact
Sorted range or monotonic answer	safe binary search

Required boundary tests

- zero and one;
- negative values where allowed;
- Integer.MIN_VALUE and Integer.MAX_VALUE;
- Long.MIN_VALUE and Long.MAX_VALUE;
- empty, null, sign-only, invalid digit, and leading zeros;
- exact square and values immediately around it;
- exact power of two and neighbors;
- one operation that fits and one that overflows;
- modulus one, negative dividend, and invalid modulus;
- equal numeric strings with different leading zeros.

16.16 Readiness assessment

Score the completed work out of 100:

Area	Points	Full-credit condition
Conceptual C01-C30	30	Correct reason, not only conclusion
Output O01-O20	20	Exact output or exception plus type reasoning
Debugging D01-D20	20	Failing input, violated contract, and repair
Short coding S01-S20	20	Compiles, correct boundaries, complexity stated
Medium M01-M10	10	Correct invariant and implementable design

Interpretation

- **90-100: Interview ready for this volume.** Begin timed mixed practice and the next series volume.
- **75-89: Nearly ready.** Rework every missed boundary and repeat the assessment within three days.
- **60-74: Partial foundation.** Revisit the relevant chapters, then implement the short set again without notes.
- **Below 60: Rebuild before adding topics.** Focus on types, overflow, digit traversal, base conversion, and GCD/root invariants.

Mandatory quality gates

A high score is not sufficient if any of these statements is false:

- [ ] I state the input contract before coding.
- [ ] I can explain every numeric type in my solution.
- [ ] I test zero, signs, and fixed-width boundaries.
- [ ] I detect overflow before trusting a result.
- [ ] I state time and space complexity with the correct input variable.
- [ ] I can dry-run the invariant aloud.
- [ ] I do not use BigInteger merely to hide a simpler streaming invariant.
- [ ] I do not avoid BigInteger when the exact arbitrary-precision value is genuinely required.
- [ ] I can complete all five follow-up chains without losing the original contract.

Five-minute oral readiness drill

Without notes, explain:

1. safe reverse integer;
2. generic base parsing with overflow;
3. prime checking through a division boundary;
4. safe GCD and LCM;
5. huge decimal string modulo;
6. integer square root;
7. Java overflow before widening;
8. negative remainder and floorMod;
9. boxed equality and null unboxing;
10. comparator overflow.

If any explanation takes more than 30 seconds to reach the invariant, mark it for one more review cycle.

RECAP | 16.17 Chapter summary

- Interview readiness depends on retrieval, implementation, debugging, and explanation.
- Numeric correctness begins with operand types and input contracts.
- The same small set of invariants solves a large family of number problems.
- Delayed answers reveal whether knowledge was recalled or merely recognized.
- Boundary tests are part of the solution, not optional polish.
- Follow-up chains test whether the invariant survives changing constraints.

TRY IT | 16.18 Final revision checklist

- [ ] I attempted all 30 conceptual questions before reading answers.
- [ ] I predicted all 20 code outputs before running them.
- [ ] I diagnosed all 20 debugging tasks with failing inputs.
- [ ] I implemented all 20 short exercises.
- [ ] I designed all 10 medium solutions.
- [ ] I rehearsed all 5 follow-up chains.
- [ ] I scored the readiness assessment honestly.
- [ ] I can use the cheat sheet as a prompt, not as a substitute for reasoning.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: retype the digit and conversion templates
- Interview Core: solve the 52-implementation catalog by invariant
- SDE-2 Follow-up: defend contracts, types, validation, and overflow behavior
- Challenge: complete the mixed revision bank and follow-up chains under time

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 02 - Number Systems and Math Foundations for DSA Interviews

Series index

Next book: DSA 04 - Bit Manipulation in Java

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 03 of 17 - Study Step 03B. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.